

Deep reinforcement learning applied to a sparse-reward trading environment with intraday data

Lucas de Azevedo Takara^a, André Alves Portela Santos^b, Viviana Cocco Mariani^{a,c} and Leandro dos Santos Coelho^{a,d}

^aDepartment of Electrical Engineering, PPGEE, Federal University of Parana, UFPR, Avenida Coronel Francisco Heráclito dos Santos, Curitiba, 81530-000, Parana, Brazil

^bCUNEF Universidad, Calle Almansa 101, 28040, Madrid, Spain

^cMechanical Engineering Graduate Program, PPGEM, Pontifical Catholic University of Parana, PUCPR, Imaculada Conceição, Curitiba, 80215-901, Parana, Brazil

^dIndustrial and Systems Engineering Graduate Program, PPGEPS, Pontifical Catholic University of Parana, PUCPR, Imaculada Conceição, Curitiba, 80215-901, Parana, Brazil

ARTICLE INFO

Keywords:

Quantitative Trading
Deep Reinforcement Learning
Deep Q-learning
Sparse-reward Trading Environment
Financial signal processing

ABSTRACT

Deep reinforcement learning (DRL) achieved significant progress in several areas enabling computers to perform complex decision-making tasks. Applied to quantitative trading, DRL trading agents can optimize their decisions during distinct market scenarios to reach a profitable strategy by learning from previous experiences. This work proposes a trading system containing a variant of the Deep Q-Network (DQN) algorithm as its main component called Extended Trading DQN (ETDQN). It can adapt its learning to trade across numerous market-behavior moments, receiving feedback only when a trade is liquidated, differently from recent approaches that provide constant feedback. Based on distributional learning and other independent extensions suggested by the DRL community, the algorithm optimizes its decision-making process by replaying prioritized experiences containing different sub-goals, assisting the agent in collecting the maximum value of profits and removing the need for fine-tuning rewards. ETDQN learned to trade on three distinct financial time series signals, identifying trading opportunities during volatile price moments. The algorithm showed more aggressive behavior regarding the volatility of its annual returns than the Trading DQN benchmark, achieving a 1.46 and 7.13 better performance with respect to average daily cumulative returns in Western Digital Corporation and Cosmos cryptocurrency historical market data.

1. Introduction

Quantitative trading is an automatic trading execution of a financial asset that utilizes mathematical models to make trading decisions (Kissell, 2021). Quantitative hedge funds use these algorithms to maximize profits due to their ability to learn from historical data.

✉ lucastakara@ufpr.br (Lucas de Azevedo Takara); andre.santos@cunef.edu (André Alves Portela Santos); viviana.mariani@pucpr.br (Viviana Cocco Mariani); leandro.coelho@pucpr.br (Leandro dos Santos Coelho)

Traders built technical analysis-based algorithms to execute actions based on pre-determined conditions using technical indicators. However, these systems have poor generalization ability and susceptibility to environmental change.

Reinforcement learning (RL) is an emerging sub-field of machine learning that enables an agent to learn in an interactive environment through trial and error, using feedback from its actions and experiences. The agent aims to improve its policy to acquire better rewards.

Deep reinforcement learning (DRL) is a combination of reinforcement learning and deep learning techniques that achieved significant progress in several areas, such as gaming (Mnih et al., 2015) and control (Levine et al., 2016), enabling computers to perform complex decision-making tasks. DRL also has achieved significant performance in trading tasks seeking to maximize profits by identifying possible trading opportunities.

Even though there exist publications about DRL-based trading systems in the literature, many of them, such as Li et al. (2019), Théate and Ernst (2021), and Shavandi and Khedmati (2022) are based on the assumption of a constant feedback to the agent due to the complicated reward tuning required to guide the algorithm towards a lucrative policy with non-frequent information.

However, providing feedback at every iteration may lead to a lack of exploration of the environment, restraining the agent from generalizing its decision-making process. In addition, financial markets do not generate meaningful information constantly. Therefore the algorithm should be trained to receive a contribution only during the end of an event, in this case, at the end of a trade.

This work proposes to implement a Deep Q-Network (DQN) algorithm variant named Extended Trading DQN (ETDQN). This variant can generalize its learning to trade across numerous market-behavior moments, receiving a simple exponential profit-and-loss reward at the end of a trade.

ETDQN extends current value-based DRL approaches adapted to trading, such as Théate and Ernst (2021), by adopting distributional learning instead of the standard expectation. Bellemare et al. (2017) suggested that this approach is beneficial since other statistics from the approximated distribution can be learned and optimized besides the mean.

The algorithm also incorporated both prioritized (Schaul et al., 2016) and hindsight (Andrychowicz et al., 2017) experience replays to sample meaningful experiences, according to prioritization. Each experience

tuple contains different sub-goals to assist the model in optimizing its decisions to collect the maximum of profits and removes the need for complex reward tuning.

To improve exploration and learning even further, ETDQN adopted the dueling neural network architecture (Wang et al., 2016) in combination with noisy linear layers (Fortunato et al., 2018). Noisy linear layers add parametric noise to the artificial neural network, where learned perturbations drive exploration, and the dueling architecture identifies which states are valuable without the need to understand the effect of each action for each state.

Finally, instead of using the common sampling approach based on regular time intervals, we adopted a market activity-driven sampling technique proposed by de Prado (2018). This method provides information to the agent at the same pace that the market processes information, not oversampling information during low activity or undersampling during high activity, as opposed to the standard time sampling approach.

ETDQN was compared against the classical benchmarks: *Buy-and-Hold* (B&H) and *Sell-and-Hold* (S&H). In addition, a strategy named Random Action which chooses its actions arbitrarily based on a discrete uniform distribution, was implemented and compared to the algorithm to determine if the model performs better than random.

Finally, ETDQN is compared to the Trading DQN (TDQN) algorithm. Proposed by Théate and Ernst (2021), TDQN was implemented according to the same instructions described in the original paper. The performance comparison considers cumulative returns, the 6-month rolling Sharpe ratio, maximum drawdown, and annual average return (AAR).

In summary, the contributions of this paper are:

- Propose an event-driven intraday trading system that is deployable on fundamental trading markets and is profitable regarding cumulative and annual returns in the long term.
- Explore a pre-processing sampling step that generates information according to market activity, contrasting the standard time sampling approach, and analyze its properties.
- Implement a DQN-variant that can identify and take advantage of trading opportunities by receiving non-frequent feedback from the environment and does not need complicated reward tuning.

The remaining parts of this paper are organized as follows: In Section 2, we provide an overview of the required preliminaries and background concepts about RL applied to trading. Section 3 presents a systematic

literature review about applying DRL algorithms on trading tasks. In the following, Section 4 describes the adopted data gathering, cleaning, pre-processing procedures, and exploratory data analysis. Then, Section 5 provides details of our experimental settings and methodology. Section 6 contains the presentation and discussion of the results achieved by the ETDQN algorithm compared to other benchmarks. Finally, Section 7 concludes this paper with final remarks and future directions.

2. Background

This section reviews some reinforcement learning concepts and DQN extensions necessary to build and evaluate the proposed ETDQN trading strategy and the used evaluation metrics that assess the algorithm's performance.

2.1. Deep Q-Networks

DQN is an algorithm that aims to approximate a state-action value function in a Q-Learning framework using a deep neural network. Introduced by Mnih et al. (2015), it was the first deep learning model to successfully learn to control policies directly from high-dimensional sensory input using RL.

The algorithm incorporated a nonlinear neural network function approximator with weights θ called Q-networks to overcome the limitation of small state and action spaces in Q-tables. The main features introduced to the algorithm to stabilize learning were Fixed Q-targets and Experience Replay. Q-learning updates its estimated action values based on another estimated value; hence this mechanism may potentially lead to harmful correlations. DQN solves this issue using a target neural network containing fixed Q-targets. This network has the same architecture as the Online Q-network, except it keeps its θ_i^- weight parameters fixed, and after τ steps, they are updated by copying the online Q-network weights θ_i . Equation 1 represents the online Q-network loss calculation.

$$L_i(\theta_i) = \mathbb{E}_{(s,a,r,s') \sim U(D)} [Y^{DQN} - Q(s, a; \theta_i)]^2, \quad (1)$$

where the subscript of the expected value operator ($\mathbb{E}$) $(s, a, r, s') \sim U(D)$ means that the variables state s , action a , reward r , and next state s' in experience tuple (s, a, r, s') are uniformly drawn from the experience replay buffer. $\mathbb{E}$ is the expectation symbol, $Y^{DQN} = r + \gamma \max_{a'} Q(s', a'; \theta_i^-)$ is the temporal difference updated action value target originated from the target neural network, with r being the immediate reward, γ the

discount factor to take most recent experiences into relevance, and $\max_{a'} Q(s', a'; \theta_i^-)$ is the maximum valued action a' taken in next state s' , with respect to the Target network with weights θ_i^- in iteration i . Lastly, $Q(s, a; \theta_i)$ is the action value originated from the online Q-network, parameterized by weights θ_i .

2.2. DQN Extensions

This subsection describes some DQN extensions proposed by the DRL community. Double DQN, Distributional DQN, Dueling neural networks, Noisy Linear layers, Prioritized and Hindsight experience replays are explained.

2.2.1. Double DQN

The standard DQN algorithm utilizes the same weights to select and evaluate an action. This makes the agent more likely to select overestimated values under certain conditions, such as when training starts while Q-values are still evolving, resulting in overoptimistic value estimates. To reduce the overestimation problem, Double DQN decomposes the $\max$ operator in the target function into action selection and action evaluation.

$$Y^{DoubleDQN} = r + \gamma \max_{a'} Q((s', a'; \theta_i); \theta_i^-), \quad (2)$$

where r is the reward, γ is the discount rate, i is the respective algorithm iteration, $\max_{a'} Q((s', a'; \theta_i); \theta_i^-)$ is the chosen Q-value generated by the online Q-network with weights θ_i , having as input the next state s' , taking action a' that results in the maximum ($\max$) value, and having the target network with weights θ_i^- as the evaluator. The loss of the algorithm is calculated as:

$$L_i(\theta_i) = \mathbb{E}_{(s,a,r,s') \sim U(D)} [Y^{DoubleDQN} - Q(s, a; \theta_i)]^2, \quad (3)$$

where $Y^{DoubleDQN}$ is the target, and the Q-value $Q(s, a; \theta_i)$ is generated by the online artificial neural network with weights θ_i in state s , and taking action a .

2.2.2. Distributional DQN

Distributional DQN was proposed to approximate the distribution of returns instead of the expectation. The algorithm modeled these distributions with probability masses placed on discrete support z , parameterized by $N_{atoms} \in \mathbb{N}^+$, given by

$$z_i = V_{min} + i\Delta z, \quad (4)$$

where $V_{min} \in \mathbb{R}$ is the minimum (starting) value of the support vector z_i whose values are evenly spaced, i is the position of the support z_i . Δz is the minimum value that is added to each position of vector z defined by:

$$\Delta z = \frac{V_{max} - V_{min}}{N_{atoms} - 1}, \quad (5)$$

where N_{atoms} is the number of atoms, $V_{max} \in \mathbb{R}$ is the maximum value of the discrete support and V_{min} is described by the previous equation.

Given an approximated distribution d_t at time-step t , defined on support z , containing the probability mass $p_\theta^i(s, a)$ on each atom i , such $d_t = (z, p_\theta^i(s, a))$. The objective is to update weights θ , such that d_t matches the actual distribution of returns.

In order to learn the probability masses, new support z should be constructed in every time t and Kullbeck-Leibler divergence D_{KL} between d_t and the target distribution $d'_t = (R_{t+1} + \gamma z, p_\theta^-(s', a^*))$ is minimized:

$$D_{KL}(\Phi_z d'_t || d_t). \quad (6)$$

Φ is a L2-projection of the target distribution onto the support vector z , R_{t+1} is the reward at time-step $t + 1$, γ is the discount factor multiplied by support vector z , $p_\theta^-(s', a^*)$ are the probability masses for next state s' taking the greedy action a^* with respect to the mean action values.

2.2.3. Dueling Network Architecture

The dueling network architecture was proposed to learn which states are (or are not) valuable without understanding the effect of each action for each state. The architecture consists of two linear stream layers such that each has the capability of providing separate estimates of the value and advantage functions. Lastly, both streams are combined to produce a single Q -value for each action.

Figure 1a shows the typical single-stream architecture adopted during the classical DQN algorithm. It consists of linear layers, followed by the rectified linear unit activation functions (RELU). A softmax activation function in the last linear layer normalizes the network's output to a probability distribution over the state-action value function.

Figure 1b shows the dueling neural network architecture. The architecture shares a common module at the beginning composed of linear layers followed by RELUs. The network is separated into two streams representing the state-value $V(s)$ and advantage $A(s, a)$ functions. At the end of the architecture, both streams are combined by an aggregating layer followed by a softmax activation function to produce the probabilities over the estimated $Q(s, a)$ state-action value function.

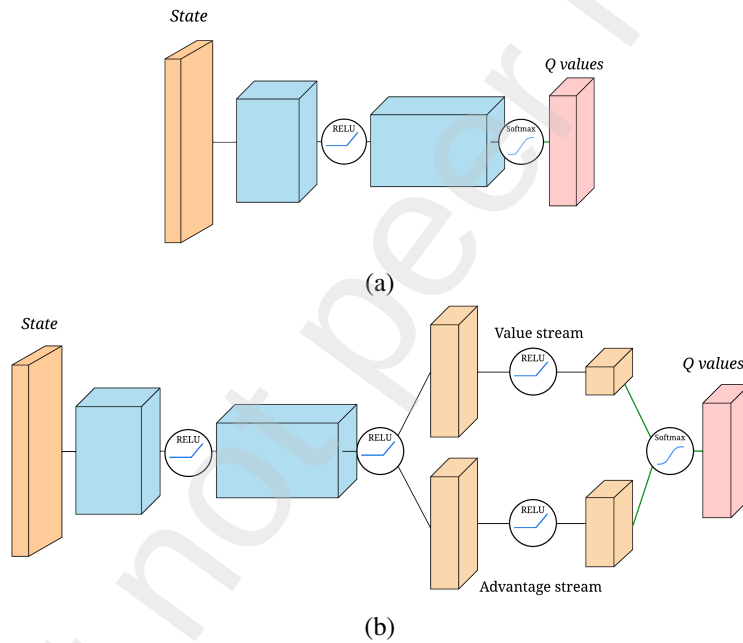


Figure 1: Neural network architectures: (a) Typical feed-forward neural network architecture adopted in DRL, (b) Dueling neural network architecture.

Equation 7 calculates the Q-function separating the advantage and state value functions:

$$Q(s, a; \theta, \alpha, \beta) = V(s; \theta; \beta) + (A(s, a; \theta, \alpha) - \frac{1}{|A|} \sum_{a'} A(s, a'; \theta, \alpha)), \quad (7)$$

where s is a state s , a is an action a , $V(s; \theta; \beta)$ is the state-value function in state s with respect to the parameters θ from the first shared linear layer of the network and β are the parameters from the value-stream only. $A(s, a; \theta, \alpha)$ is the advantage function of the action a in state s with respect to the parameters θ and

α , which are the parameters from the Advantage stream branch of the network. $\frac{1}{|A|} \sum_{a'} A(s, a'; \theta, \alpha)$ is the average of the advantage values produced in state s taking the best evaluated action a' with respect to the parameters θ and α . $\sum$ represents a sum of multiple terms;

2.2.4. Noisy Linear Layers

Noisy linear layers replace the conventional exploration ϵ -greedy heuristic for a noisy linear layer that adds parametric noise to its weights where learned perturbations of the network weights are used to drive exploration.

The feature's main idea is that a single change to the weights vector may influence a consistent state-dependent shift in policy over multiple time steps. A noise distribution samples the perturbations, and variance is a parameter that can be considered as the energy of the injected noise. These variance parameters are learned using gradients from the reinforcement learning loss function alongside the other parameters of the agent. Suppose a linear layer from a neural network with p inputs and q outputs given by:

$$y = wx + b, \quad (8)$$

where $x \in \mathbb{R}^p$ is the layer's input, $w \in \mathbb{R}^{q \times p}$ is the neural network's weight matrix, and neural network's $b \in \mathbb{R}^q$ is the bias. Its corresponding noisy linear layer is defined by:

$$y = (\mu^w + \sigma^w \odot \epsilon^w)x + \mu^b + \sigma^b \odot \epsilon^b, \quad (9)$$

where $\mu^w \in \mathbb{R}^{q \times p}$ and $\sigma^w \in \mathbb{R}^{q \times p}$ are the learnable parameters from the weight matrix, $\mu^b \in \mathbb{R}^q$ and $\sigma^b \in \mathbb{R}^q$ are learnable parameters for the neural network's bias. $\epsilon^w \in \mathbb{R}^{q \times p}$ and $\epsilon^b \in \mathbb{R}^q$ are noise random variables from the weight matrix and bias respectively, and $\odot$ is the outer product operator.

2.2.5. Prioritized Experience Replay

Prioritized Experience Replay is a technique that modifies the sampling mechanism of the standard DQN algorithm, which samples transitions uniformly at random. The main idea of the technique is to prioritize transitions that maximally reduce the global loss in its current state that improves on uniform random replay.

$$P(i) = \frac{p_i^\alpha}{\sum_k p_k^\alpha}, \quad (10)$$

where $p_i > 0$ corresponds to the priority of experience i . α how much prioritization is used. Suppose $\alpha = 0$, the probability of sampling the transition corresponds to the uniform case. $\sum_k p_k^\alpha$ is the sum ($\sum$) of all experiences priorities in the k mini-batch.

$$w_i = \left(\frac{1}{N} \times \frac{1}{P(i)} \right)^\beta, \quad (11)$$

where $P(i)$ is the probability of sampling a transition i , β is a compensation factor, and N is the amount of experiences. The authors define a schedule on β exponent that reaches one only at the end of the learning exploiting the flexibility of annealing the amount of importance-sampling correction over time.

2.2.6. Hindsight Experience Replay

Hindsight experience replay (HER) was proposed to deal with sparse rewards. The main idea of HER is to replay each episode with a different goal than the one the agent was trying to achieve, generalizing not just over states $s \in S$ but also over goals $g \in G$. In the algorithm's simplest form, the goal g , achieved in the final state of the episode, is concatenated with the current state s_t ($s_t \parallel g$) and serves as input to the agent, where $\parallel$ denotes concatenation. Then, the action a_t is executed at time t , and it observes the next state s_{t+1} , and the next state is also concatenated with the goal g ($s_{t+1} \parallel g$) and stored as an experience tuple in the replay buffer ($s_t \parallel g, a_t, r_t, s_{t+1} \parallel g$).

2.3. Activation Functions

This subsection describes the activation functions that compose the architecture of the adopted neural networks.

2.3.1. Rectified Linear Unit

The Rectified Linear Unit activation function has become the default activation function for many types of neural networks. It is defined as:

$$f(x) = \begin{cases} x & \text{if } x > 0, \\ 0 & \text{otherwise.} \end{cases}, \quad (12)$$

where $f(x)$ is the output of the function, x is the input. This function will output the input directly if it is positive; otherwise, it will output zero.

2.3.2. Softmax

The function takes a vector $\mathbf{z}$ of K real numbers and normalizes it into a probability distribution consisting of probabilities proportional to the exponential of the input numbers.

$$\sigma(\mathbf{z})_i = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}. \quad (13)$$

In this equation, each vector element z_i , with $i = 1, \dots, K$. It serves as an exponent for Euler's number e . In the following, the resulting value is divided by the sum $\sum$ from $j = 1, \dots, K$ of each element of the vector $\mathbf{z}$, also serving as an exponent to e .

2.4. Evaluation Metrics

This subsection describes the evaluation metrics on which the algorithm is evaluated. Some essential metrics for trading stocks include the Sharpe ratio, simple returns, cumulative returns, and Maximum drawdown. The following subsections describe the metrics previously mentioned.

2.4.1. Simple Returns

Simple returns on an investment are the amount the investment has gained or lost over time, independent of the time involved. Arratia (2014) defines simple returns calculation. The τ -period simple return at time t , $R_t(\tau)$, is the rate of change in the price obtained from holding the asset from time $t - \tau$ to time t .

$$R_t(\tau) = \frac{P_t - P_{t-\tau}}{P_{t-\tau}} = \frac{P_t}{P_{t-\tau}} - 1, \quad (14)$$

where P_t is the price of an asset at time t , in a time scale τ .

2.4.2. Cumulative Returns

Cumulative Returns, also known as multi-period returns, show the evolution of the investment considering its historical path. Arratia (2014) defines that the τ -period simple return $R_t(\tau)$ at time t is equals the product of τ one-period simple gross returns at times $t + 1$ to t . Equation 15 shows the proof of proposition 14:

$$R_t(\tau) + 1 = \frac{P_t}{P_{t-\tau}} = \frac{P_t}{P_{t-1}} \cdot \frac{P_{t-1}}{P_{t-2}} \dots \frac{P_{t-\tau+1}}{P_{t-\tau}}$$

$$= (1 + R_t) \cdot (1 + R_{t-1}) \cdots (1 + R_{t-\tau+1}). \quad (15)$$

2.4.3. Sharpe Ratio

The Sharpe ratio measures the risk-adjusted return of a financial portfolio. In 1966, Sharpe (1994) proposed this risk/reward ratio to capture in a single number. Equation 16 defines the Sharpe ratio:

$$S_a = \frac{\mathbb{E}[R_a - R_b]}{\sigma_a}. \quad (16)$$

The quality of an investment is defined as the expected value $\mathbb{E}$ of the asset return R_a subtracted by the risk-free return R_b , divided by the standard deviation of the asset σ_a . The lower the standard deviation, the less risk and the higher the Sharpe ratio; the opposite is true.

2.4.4. Maximum Drawdown

Maximum Drawdown (MDD) is the highest peak-to-valley decline (usually quoted as a percentage) of an investment during a specific holding period. Drawdowns can help to determine an investment's financial loss risk by looking into the past. Choi (2021) defines the Maximum Drawdown as

$$MDD(T) = \max_{\tau \in (0, T)} \left[\max_{t \in (0, \tau)} (P(t) - P(\tau)) \right], \quad (17)$$

with the inner equation part computing the drawdowns that are calculated by the *max* of the difference of the logarithmic-price $P(t)$ at time t ranging from 0 to τ , and $P(\tau)$ the logarithmic-price at the period τ , and the outer part takes the *max* of the drawdowns, which is the maximum drawdown.

3. Related Works

This section performs a systematic review of the literature (SLR) about DRL applied to trading tasks. Aiming to have an overview of the existing literature on the topic, reinforcement learning, deep learning, and deep reinforcement learning papers related to the previously mentioned topic were collected, analyzed, and discussed.

In addition, the following literature review explored Scopus, Web of Science, and Google Scholar academic research databases, seeking to retrieve current relevant information about the topic, limiting its search space from 2017 to 2022.

3.1. Systematic Literature Review

Adopting Kitchenham and Brereton (2013) statement guidelines, the SLR was divided into a primary three-stage process: planning (definition of the research questions), conduction (inclusion and exclusion criteria definitions, academic research databases search strategy, paper selection validation, and data extraction), and lastly, a summary containing the extracted results is presented.

This systematic literature review addressed the following research questions:

- RQ1. What are the main reinforcement learning methods applied to quantitative trading?
- RQ2. What are the main performance metrics used on these algorithms?
- RQ3. What are the main markets to which the models have been applied?

Once the research questions were established, the planning stage was completed and followed by conduction. In this step, the following inclusion criteria were adopted:

- (a) Primary studies publications;
- (b) Published within the last five years;
- (c) English language publications;
- (d) Journal and conference published papers;
- (e) Publications relevant to deep learning, reinforcement learning, and deep reinforcement learning topics related to quantitative trading tasks.

The exclusion criteria were set to identify publications, which were eventually removed from the research study. The following criteria are adopted:

- (a) Secondary studies publications;
- (b) Technical reports and pre-print papers;
- (c) Non-English language paper publications;
- (d) Publications that do not approach reinforcement learning or deep reinforcement learning methods related to quantitative trading tasks;

(e) Non-downloadable publications on their respective research database.

In sequence, through *Coordenação de Aperfeiçoamento de Pessoal de Nível Superior* (CAPES (2023)), both Scopus and Web of Science research databases were accessed, and a keyword-based search query was put into their respective advanced document search tab. These databases contain traditional publishers such as Elsevier, Springer Link, and many others. In order to correctly generate the search query input, OR and AND operators were included. The first operator was placed between search groups, whereas the remaining one was put between synonyms. The following query search was generated:

("Finance Trading" OR "Quantitative Trading" OR "Algorithmic Trading" OR "Trading" OR "Strategy Trading")) AND (("Reinforcement Learning" OR "Deep Learning" OR "Deep Reinforcement Learning")

In the first stage, all fields were included (no date range or language limitations). A total of number 808 publications was retrieved. After that, papers were selected accordingly to the inclusion criteria, and the search space was constrained to the article title, abstract, and keywords only. A total number of 376 publications were selected. Then, the selected papers from stage 2 were evaluated accordingly to the exclusion criteria. A total of 1 article was not English written, 47 were not published between 2017-2022, and 42 were duplicated.

Analyzing their respective title, 44 were off the review's scope. Some of them included DRL methods but were not applied to trading tasks. In stage 4, papers were selected and removed based on their respective title, abstracts, and conclusion analysis, resulting in 188 papers. Lastly, articles were excluded accordingly to their complete analysis. 158 publications were selected.

Figure 2 shows the amount of RL-related publications applied to trading tasks per year between 2017 - 2022. A total of 90 journals and 65 conference publications were selected, with the majority published in 2021. During this year, a total amount of 40 journal papers and 21 conference articles were published. The bar chart indicates an ascending slope during the years, meaning the following year has a more significant publication amount than its precedent, except in 2022.

Figure 3 shows the number of RL-related publications retrieved by research databases separately: Elsevier, Institute of Electrical and Electronics Engineers Xplore digital library (IEEE Xplore), Springer, and Hindawi, among others. In terms of percentage with respect to the amount of RL-related works by publication types, 56.96% of the publications come from journals and 43.04% from conferences. Elsevier

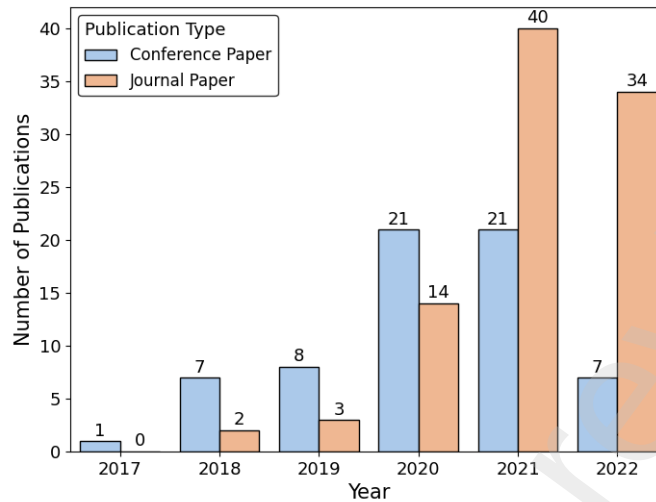


Figure 2: Number of publications per year of DRL algorithms applied to quantitative trading tasks between 2017 and 2022.

research database had the most significant contribution with 39 papers, followed by IEEE Xplore and Multidisciplinary Digital Publishing Institute (MDPI), having 30 and 26 publications, respectively.

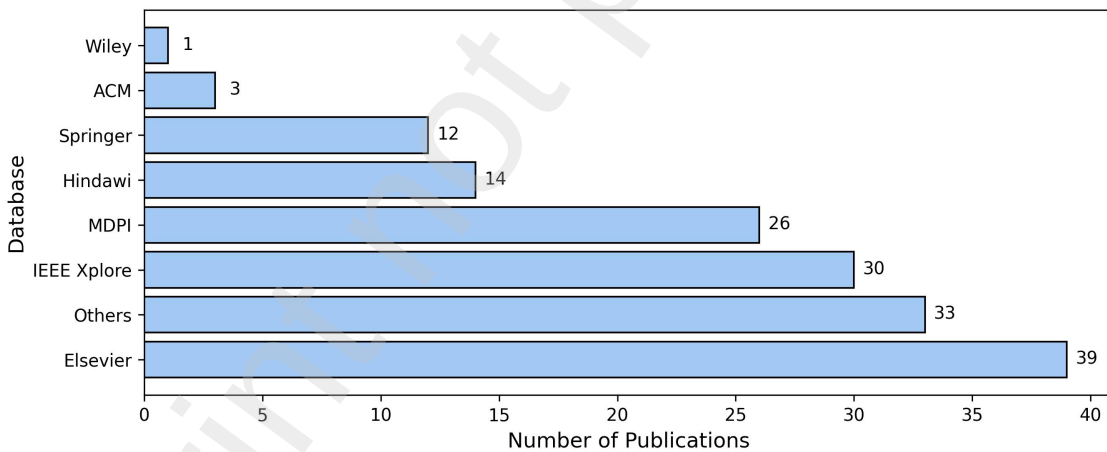


Figure 3: Number of publications by research databases of DRL methods applied to quantitative trading tasks between 2017 and 2022.

3.1.1. Journal Publications

Li et al. (2019) extended both value-based DQN and actor-critic Asynchronous Advantage Actor Critic (A3C) DRL algorithms to the trading market and utilized an long short-term memory (LSTM) module

to capture temporal patterns based on market observations. Stacked denoising autoencoders are used for filtering using open, high, low, close (OHLC) prices, trading volume, and specific trading indicators such as moving average convergence/divergence (MACD), moving average (MA), exponential moving average (EMA), average true range (ATR), and price rate of change. Sharpe ratio is set as the reward function that is provided constantly. A novel positions-embedded action space is presented, allowing the agent to learn to control positions (holding more positions in a bull market while decreasing positions in a bear market).

Wu et al. (2020) proposed a couple of adaptive stock trading strategies, incorporating gated recurrent units on both DQN and Deep Deterministic Policy Gradient (DDPG) algorithms. By using MA, EMA, MACD, volatility rank, and on-balance volume indicators, the agents are compared to the turtle trading and the state-of-the-art direct reinforcement learning strategies. The previous comparison shows an increase in stability using the Sortino ratio and rate of return as performance metrics regarding the DRL-based strategies.

Théate and Ernst (2021) introduced a DQN-inspired algorithm denominated Trading Deep Q-Network (TDQN) that contains a modified version of the standard DQN, including the Double DQN algorithm, a feed-forward network architecture, Adam optimizer, and the use of Huber loss. Training in daily historical data, TDQN is applied to various indexes such as Dow Jones, National Association of Securities Dealers Automated Quotations (NASDAQ), and Standard and Poor's 500 (S&P500), as well as other stocks involving technology and financial services. To evaluate the algorithm's performance, B&H, S&H, Trend following with moving averages, and Mean reversion with moving averages strategies are compared to it by using the Sharpe ratio, profit & loss, Annualized returns, Annualized volatility metrics among others. The authors used dense returns as rewards to the agent.

Carta et al. (2021) presented a multi-DQN algorithm that is applied to several real-world trading scenarios, such as the S&P500 future market and the J.P. Morgan. The author includes a pre-processing step based on generating meta-features after converting price time series into Gramian Angular Field images. After that, a DRL-based meta-learner processes the signals provided by a Convolutional Neural Network (CNN) to generate final trading decisions. Sharpe ratio, profit & loss, annualized return, volatility, profitability, profit and loss, and Sortino ratios, and maximum drawdown are performance metrics. In addition, a couple of different ensembling techniques (stacking and majority voting) are used to maximize the robustness and stability of the strategy. In the end, the benchmark B&H strategy is compared to the proposed model.

Sagiraju and Mogalla (2022) propose a framework that includes four DRL algorithms, namely Advantage Actor-Critic (A2C), Proximal Policy Optimization (PPO), DDPG, and DQN that used historical stock and Twitter market sentiment data consisting of Dow Jones and S&P 500. These are compared using the financial metrics, Sharpe ratio, Sortino ratio, maximum drawdown, cumulative returns, annual volatility, and annualized investment return. The reward system was not explicitly mentioned during the work.

Lastly, Koratamaddi et al. (2021) also propose a sentiment-aware approach as an extension to the adaptive DDPG algorithm. The model adjusts the amplitude of change in the Q-value per epoch, using different learning rates depending on whether the prediction error was positive or negative. In addition, the state passed to the agent is extended by including a sentiment confidence score. The reward system is fine-tuned to make it learn about the market sentiment effectively. Two components formulate the objective function: Reward by the change in portfolio value from the previous day and market sentiment confidence score. This last component is formulated by the respective Twitter and Google News mean sentiment scores.

3.1.2. Conference Publications

Jiang and Liang (2017) train a DDPG agent combined with a CNN to perform portfolio management in the cryptocurrency market. Aiming to maximize returns while restraining the risk, the author used the Sharpe ratio, maximum drawdown, final portfolio value, and standard deviation to evaluate the risk of the strategies. B&H, Best Single Asset, Uniform Constant Rebalanced Portfolio, Universal Portfolio, Online Newton Step, Passive Aggressive Mean Reversion benchmarks are compared to the proposed model.

Combining recurrent reinforcement learning (RRL) and LSTMs, Si et al. (2017) implement a multi-objective structure that measures the profit and risk separately. Sharpe ratio, annual profit, and total trading number are adopted as performance metrics, and the agent is compared with the B&H strategy benchmark and a primary RRL agent. The system is applied to three index-based future contracts traded in China stock-IF, stock-IH, and stock-IC contracts, which are three index-based futures contracts traded in China with 1-min close prices as input data.

Shin et al. (2019) propose a deep multi-modal reinforcement learning policy, combining both CNN and LSTM neural networks to a DQN-based agent. The strategy model is trained on 256 stocks listed at KOSPI, using daily OHLC prices, trading volume, moving averages, and a slow stochastic oscillator. The previous indicators are used to generate the three types of images that serve as input to the CNN. The first image combines the candlestick chart, four moving average curves (5, 20, 60, and 120 days), and a bar graph

of the trading volume. The second image combines the average directional movement index, the positive directional indicator, and the negative directional indicator. Lastly, the third image is the slow stochastic oscillator. In the end, average returns, standard deviation, maximum profits, minimum returns, and Sharpe ratio are performance metrics.

Conegundes and Pereira (2020) investigate the potential of DDPG applied to 13 different stocks listed on the Brazilian stock market index to solve the asset allocation problem. Open and close prices are used as input, and the model is compared to some stock portfolios suggested by the leading Brazilian banks and brokers during these years. The model is backtested using annual returns, cumulative returns, and maximum drawdown as performance metrics.

Yang et al. (2020) explores ensemble methods, training an agent to obtain an ensemble trading strategy using three actor-critic algorithms: PPO, A2C, and DDPG. Inheriting and integrating its best features, the strategy robustly adjusts to market situations. Applied to the Dow Jones 30 constituent stocks, the agent's performance is evaluated with the cumulative, annualized return, volatility, Sharpe ratio, and maximum drawdown performance metrics. In the end, the strategy is compared to the Dow Jones Industrial Average (DJIA) and min-variance portfolio allocation benchmarks.

3.2. Co-occurrence Network

VOSviewer software was used to provide an overview of the data synthesis. Built by van Eck and Waltman (2010), the tool allows bibliometric network visualization and construction based on bibliographic co-authorship, co-citation, and co-occurrence among similar keywords. In order to generate the co-authorship and co-occurrence networks, the information of the selected previous selected papers was exported into the ".ris" data format from the Scopus and Web of Science databases and served as input to the tool.

Figure 4 shows the keyword co-occurrence networks. By setting a minimum number of 3 keyword occurrences, from the total amount of 444 keywords, only 19 met the threshold. For each, the total strength of the co-occurrence links with other keywords is calculated, and the ones with the most significant total link are selected. VosViewer calculated the total amount of 4 clusters. It is possible to observe that Machine Learning and Deep Learning keywords were the most significant clusters. Both had 25 occurrences and a total link strength of 42 and 37, respectively. In addition, neural networks, algorithmic trading, artificial intelligence, finance, reinforcement learning, and explainable AI also contained a significant number of occurrences, each with 10, 9, 7, 7, and 6, respectively.

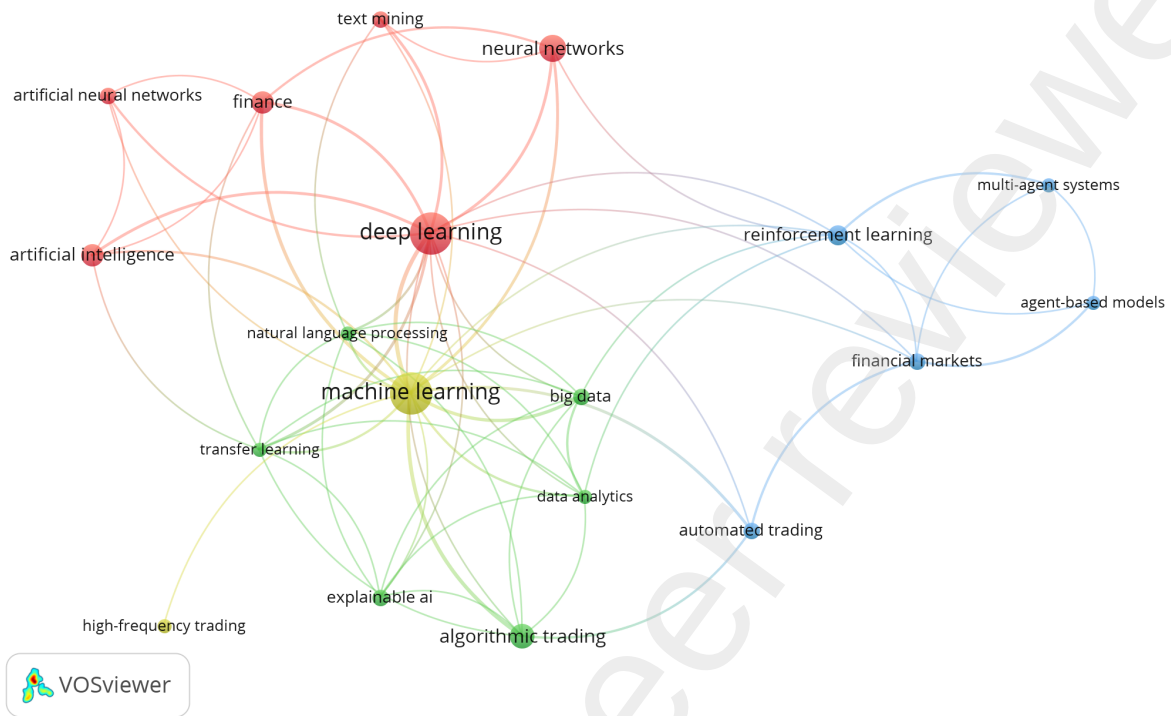


Figure 4: Co-occurrence network of DRL-related publications applied to quantitative trading tasks between 2017 and 2022.

3.3. Data Synthesis

RQ1: Through the Figure 5, it was possible to identify that the actor-critic DDPG method was the main deep reinforcement learning algorithm applied to quantitative trading tasks, having 42.1% of total citations on the DRL-related papers. DQN and PPO had 28.9% and 18.4%, respectively, and consequently, achieved the second and third positions. A2C (5.3%), A3C (2.6%), and RRL (2.6%) were also claimed to be implemented in some reviewed works.

Table 1 shows the most common performance metrics used for performance measurement purposes by the previously mentioned algorithms. A total amount of 5 papers (6.5%) claimed to use the annualized returns, eight papers (10.4%) used maximum drawdown, ten papers used Sharpe ratio (13%), and three papers (3.9%) used standard deviation to measure performance. Total profits were also used in the same proportion as the previously mentioned metric.

RQ2: According to Table 1, the Sharpe ratio was the primary metric applied to performance evaluation. 13% of all DRL-related reviewed publications claimed to have used such a metric. In addition, maximum

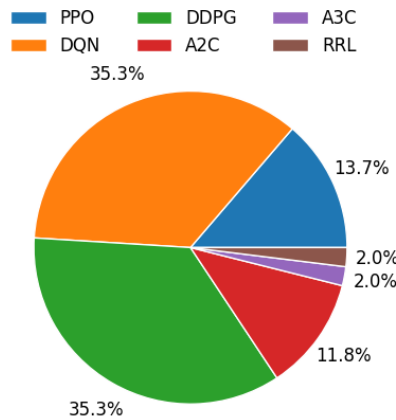


Figure 5: Appearance amount in percentage of each DRL algorithm applied to quantitative trading tasks.

Table 1

Main performance metrics applied to the reviewed works

Performance Metrics	Absolute Frequency	Percentage Frequency
Sharpe Ratio	10	13%
Maximum Drawdown	8	10.4%
Annualized Returns	5	6.5%
Standard Deviation	3	3.9%
Total Profits	3	3.9%
Others	48	62.3%
Total	77	100%

drawdown and annualized return were used, each containing 10.4% and 6.5%, respectively. Finally, total profits and standard deviation were applied and cited in 3.9% of all DRL-related reviewed works.

Table 2 shows the most common markets to which the previously mentioned works were applied. S&P500 had the absolute frequency of 7 papers, having its application on 14.3% of the RL-related works. Cryptocurrency, Dow Jones, and NASDAQ had a total amount of 3 papers (6.1%) each. Stock-IF, Stock-IC, and foreign exchange market (FOREX) had two papers (4.1%) each.

RQ3: As stated in Table 2, S&P500 was the primary market for the reviewed agents that had been applied to. 14.3% of the RL-related reviewed publications have used such market to apply their algorithms, followed by the Stock-IF with 8.2%, Cryptocurrency, Dow Jones, and NASDAQ markets that respectively had 6.1% each, and lastly, FOREX and Stock-IC markets with 4.1%.

Table 2

Main markets applied to reviewed works

Markets	Absolute Frequency	Percentage Frequency
S&P500	7	14.3%
Stock-IF	4	8.2%
Cryptocurrency	3	6.1%
Dow Jones	3	6.1%
NASDAQ	3	6.1%
FOREX	2	4.1%
Stock-IC	2	4.1%
Others	25	51%
Total	49	100%

4. Data Collection, Cleaning, Pre-processing and Exploratory Analysis

This section describes the adopted data gathering, cleaning, and pre-processing processes. First, we define the data-gathering procedure, then the outliers removal method, and finally, we conduct exploratory data analysis comparing both 1-minute time and Dollar bars' statistical properties.

4.1. Data Collection

In order to generate Dollar bars, we downloaded transaction data from the iShares Core S&P 500 ETF (iShares Core S&P 500 ETF) and Western Digital Corporation from the Kibot website, which offered free tick with bid/ask intraday data. These features were discarded, adopting prices and shares amount only. The original downloaded iShares Core S&P 500 ETF data frame contained 10,460,165 transactions corresponding to the periods of 2009-09-28 09:30:00 / 2022-12-02 16:00:00. Western Digital Corporation included 69,242,555 transactions from 2009-09-28 09:41:53 to 2022-09-21 16:00:00. Finally, the Cosmos / United States Dollar Tether (ATOM/USDT) cryptocurrency pair market data was collected via Binance spot application programming interface comprehending the period of 2019-06-05 16:35:28 / 2021-11-26 12:11:26. The data frame had 30,535,634 transactions.

4.2. Data Cleaning

The Interquartile method identifies and removes outliers to clean the data collected in the previous section according to a sliding window approach. In this procedure, the entire data frame is rearranged in ascending order and divided into four parts (quarters). The first quartile (Q_1) is the middle value between the first data sample and the median from the dataframe, the median is the second quartile (Q_2), and the third quartile

(Q_3) is the middle value between the median and the last term. Given a dataset with N observations arranged in ascending order, the quartiles are calculated by:

$$Q_1 = (N + 1) \times \frac{1}{4},$$

$$Q_2 = (N + 1) \times \frac{1}{2},$$

$$Q_3 = (N + 1) \times \frac{3}{4}.$$

In the following, the interquartile range (IQR) is calculated by:

$$IQR = Q_3 - Q_1, \quad (18)$$

followed by the lower and upper boundaries, which are defined by Equations 19 and 20, respectively.

$$Lower = Q_1 - (1.5 \times IQR), \quad (19)$$

$$Upper = Q_3 + (1.5 \times IQR), \quad (20)$$

where the Lower boundary *Lower* is Q_1 subtracted from the product of 1.5 and the interquartile range IQR and the Upper boundary *Upper* is the sum of Q_3 with the product of the interquartile range and 1.5.

In this work, we apply the IQR method with a sliding window approach. Every time a state is assembled, the data from the last seven days concerning the current time is stored in an array, and the IQR method is applied to remove outliers.

4.3. Data Pre-processing

This subsection concerns the pre-processing step that is performed after the outlier removal from the previous section. In every state assemble, transaction data collected is manipulated and transformed into Dollar bars. For comparison purposes, 1-minute bars were also generated from the same transaction data and compared to Dollar's.

Contrasting time bars, Dollar bars are not sampled in a constant pre-defined frequency, but every time a pre-defined amount of market value has been exchanged. In order to generate time bars from transactions, a constant pre-defined frequency must be chosen to be the interval in which they are generated.

The open price is equivalent to the first transaction price during that period; the high is equivalent to the transaction that achieved the highest price compared to the remaining transactions. The same opposed logic applies to the low. The close price is equivalent to the last transaction price, and finally, volume is equivalent to the sum of the amount of exchanged shares during this interval.

Figure 6 compares time and Dollar bars. Transaction data from the iShares S&P500 ETF, corresponding to the interval of 2009-09-28 09:41- 2009-09-28 10:32, were used to create both. Time bars were sampled in a 1-minute frequency, whereas Dollar bars were sampled every time market reached 500K exchanged Dollars.

It was observable that during this period, the total amount of 52 and 21 time and Dollar bars were sampled, respectively. What can be concluded is that during this period, the total amount of exchanged value of 500K USD was achieved 21 times. Noticeably, time bars undersample information during 9:41 and 9:51, in an ascending price movement, lacking show strength.

On contrast, it is observable that Dollar bars do this by generating a couple of solid green candles. It is also noticeable that between 9:51 and 10:00, time bars oversample information, indicating a price fall. In contrast, Dollar bars show that this pullback is not strong due to the exchanged value; hence goes sideways and generates weak Dollar bars. This type of bar assists the agent in understanding the relationship between market value and price movements.

4.4. Exploratory Analysis

Figure 7 shows the time series statistical properties of the iShares S&P500 ETF, including its price, logarithmic returns, amount of weekly generated Dollar bars per year, and daily volatility.

Figure 7a shows the close price feature evolution between 2009-2022. This asset shows a positive defined trend, with a few declines during 2016, 2019, and 2020. Figure 7b, returns are stationary, and a few spikes are detected on both sides of the data frame: In 2010, the ETF had its best return of 12.25% and its worst in 2020, having an -11.13% dump due to the Coronavirus pandemic (COVID-19). It is also worth mentioning a fluctuation during 2016 in which 11.2% and -8.2% returns are present. In addition, the returns have a standard deviation of 0.11%, a variance of 0.012%, and a 0.00025% mean.

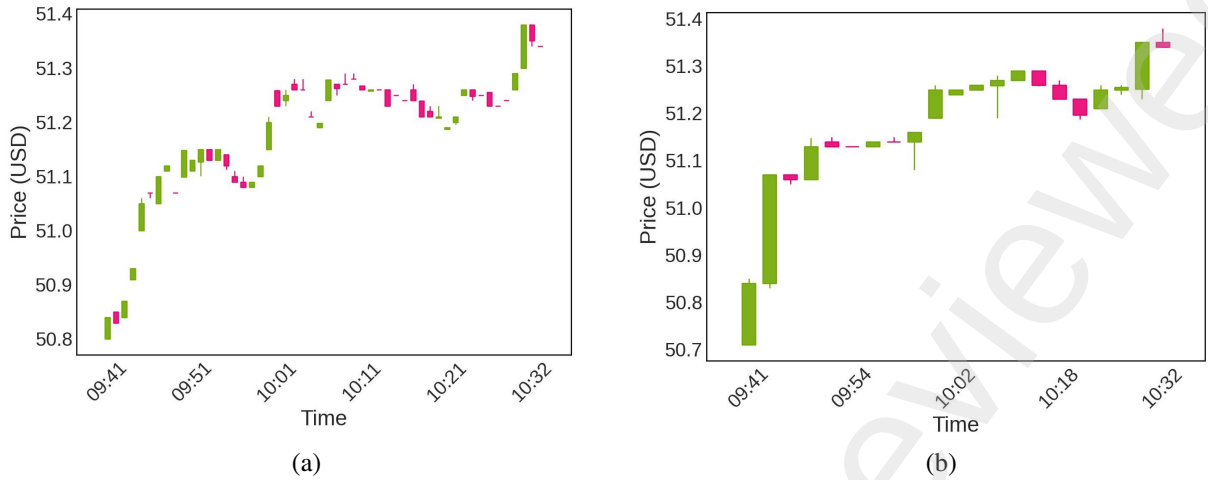


Figure 6: Bar sampling method comparison applied to the iShares S&P500 ETF data frame from 2009-09-28 09:41 to 2009-09-28 10:32: (a) time bars sampled in a 1-minute frequency, (b) Dollar bars sampled every 500K Dollars market-value exchange.

Figure 7c, shows the amount of weekly sampled bars per year in the asset. The total amount of 398,362 Dollar bars were generated against 1,081,690 1-minute time bars. 2018 had the highest exchanged market value compared to the remaining years, resulting in 56691 Dollar bars. On the other hand, 2009 had the lowest exchanged market value having 2902 bars only. This may have occurred due to missing data since the data frame started at 2009-09-28 09:30:00.

Volatility is the statistical measure of the dispersion of returns for a given asset during a period. This measure provides information about the price fluctuation around its mean in percentage. The standard deviation of returns calculates daily volatility, which is defined as:

$$\sigma = \sqrt{\frac{\sum_{i=1}^N (x_i - \mu)^2}{N - 1}} \quad (21)$$

where σ is a measure of the dispersion of a population with N elements. σ is the sum $\sum$ of the square of each value from the population x_i , from $i = 1, \dots, N$, subtracted from the population's mean μ , divided by the $N - 1$. Finally, the squared root of this value is taken.

Regarding the interpretation of the standard deviation, a couple of insights can be deduced: a high σ value means that the data points that belong to the population are dispersed; in other words, the data points are generally far from the mean. On the other hand, a low standard deviation means that the values are typically clustered close to the mean.

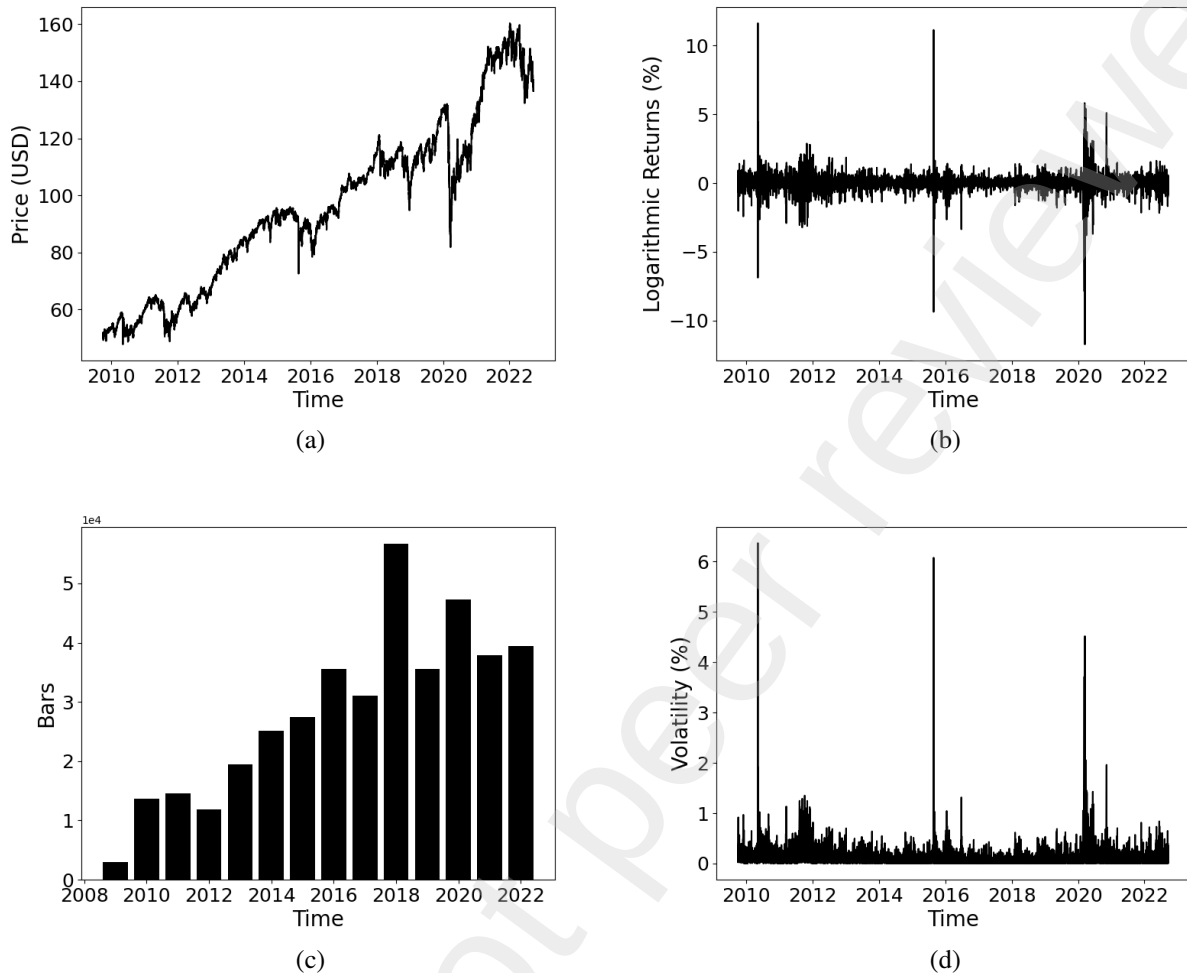


Figure 7: iShares Core S&P 500 ETF time series properties from 2009 to 2022: (a) Price series in United States Dollar, (b) Simple logarithmic returns, (c) Amount weekly Dollar bars of generated per year, and (d) Daily volatility.

Figure 7d shows the daily volatility of the iShares S&P500 ETF, having rolling standard deviations in a 7-bar window. Good-day trading opportunities are usually related to volatile moments. Three main volatility spikes were detected in 2010 (6.32%), 2016 (5.44%), and 2020 (4.3%). Moments of medium volatility, such as in 2012 and 2020, are also present in the ETF. iShares Core S&P 500 ETF does not have high volatility. Figure 8 shows time series properties generated from the Western Union Corporation data frame.

Similarly to iShares Core S&P 500 ETF, Figure 8a shows the close price feature evolution between 2010-2022. Contrary to Figure 7a, this asset is not trending; its price behaves similarly to an oscillator. Western Digital's returns have greater amplitude than the previous data frame. This can be confirmed in Figure 8b,

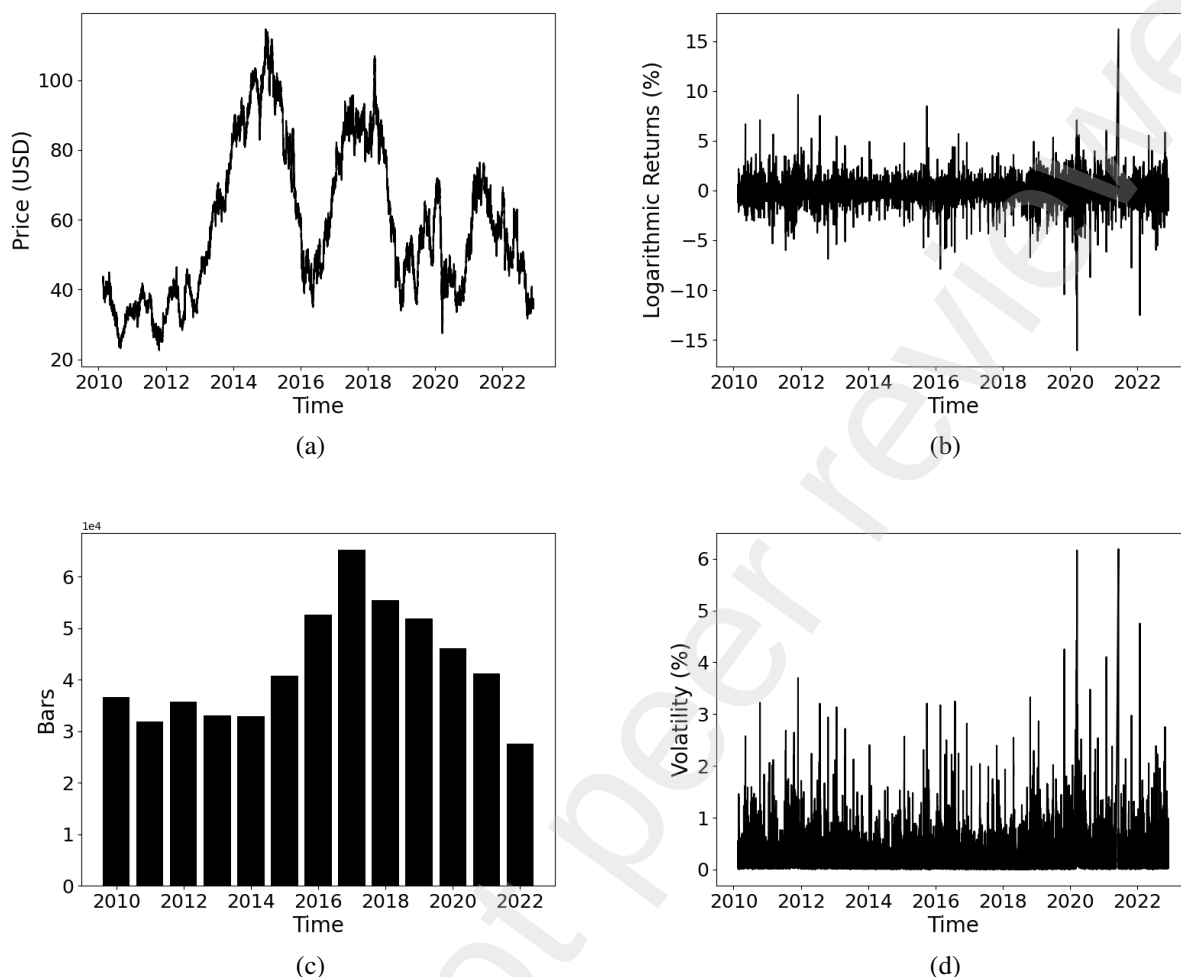


Figure 8: Western Digital Corporation time series properties from 2010 to 2022: (a) Price series in United States Dollar, (b) Simple logarithmic returns, (c) Amount weekly Dollar bars of generated per year, and (d) Daily volatility.

which shows the logarithmic returns plot. 15% both-sided returns are present during the years of 2020 and 2021, as well as -6.2% and -5.9% variations in 2022 and 2016 respectively. The logarithmic returns contain a mean of -0.000035%, a standard deviation of 0.21%, and a variance of 0.0441%.

Figure 8c shows the generated Dollar bars per week from 2010-2022. 2020 had the maximum amount of 110,186 bars, whereas its lowest value of 88,450 in 2010. The defined market value to sample a Dollar bar in Western Digital Corporation data was 1,000,000 USD. Since this asset holds more transactions than the previous one, it contains higher liquidity; 550,598 Dollar bars were generated against 1,305,281 1-minute time bars.

Lastly, Figure 8d shows the daily volatility of the asset. Compared to iShares Core S&P 500 ETF, Western Digital's close price volatility is more often. The highest volatile moments were during 2020-2022, with both variations close to 6%. The COVID-19 pandemic and the United States of America inflation increase events may explain these spikes. Finally, Figure 9 shows time series properties generated from the Binance spot ATOM/USDT cryptocurrency pair.

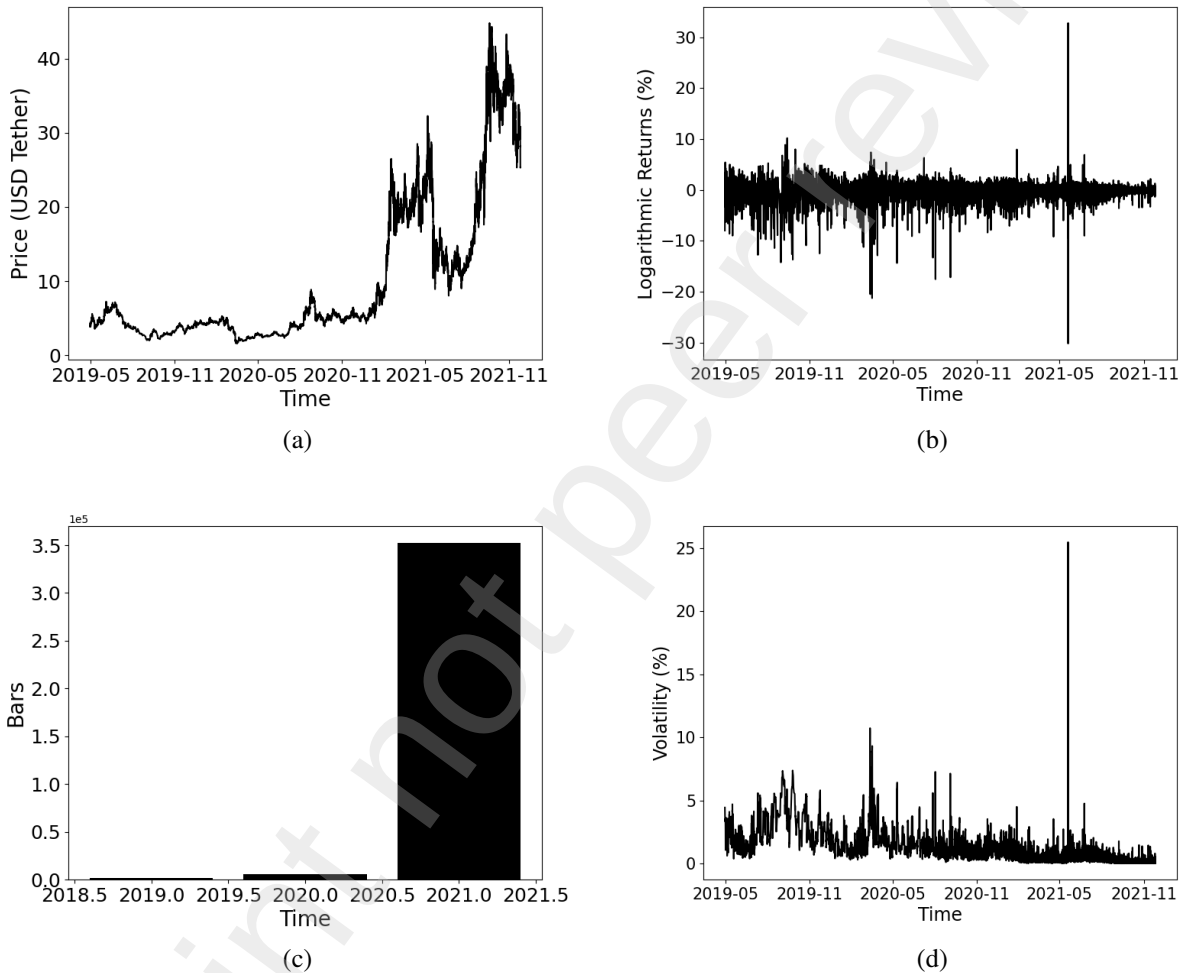


Figure 9: ATOM/USDT time series properties from 2019 to 2021: (a) Price series in United States Dollar, (b) Simple logarithmic returns, (c) Amount weekly Dollar bars of generated per year, and (d) Daily volatility.

Differently from both analyzed assets, ATOM/USDT price contains trending movements, such as in July 2021, and sideways fluctuation in October 2020. The asset shown in 9a demonstrates volatile movements. Figure 9b shows the logarithmic returns plot generated between May 2019 and November 2021. According to

it, the highest spike occurred in 2021 during a -32% drop, followed by a recovery. A few -17% approximately spikes can be detected in April 2020, as well as some 10% variations in the last quarter of 2019.

The defined market value amount to sample a Dollar bar in the ATOM/USDT data frame was 7.5K USDT. A total number of 359,786 Dollar bars were generated against 1,197,149. It is observable that the amount of time bars is 232.74% higher than Dollar ones. The cryptocurrency asset has as much data as the previous data frames because its market operates uninterruptibly. It is noticeable that in Figure 9c, the amount of the generated Dollar bars grows exponentially. The year 2021 sampled the highest amount of time bars and 2019 the smallest; each generated 475,388 and 259,249, respectively.

Figure 9d shows the volatility plot of the data frame. According to the rolling volatility, this is the most volatile asset analyzed in this master dissertation, having its maximum volatility moment evaluated at 25%. Some other 6% to 12% volatile moments are also present in April 2020. Regarding the overall trend, volatility increases during the second semester of 2019 and starts to reduce from 2021 onwards. Additionally, ATOM/USDT had a 12% volatility moment during the COVID-19 event, as well as its 25% most volatile moment in May 2021. After these events, the asset's volatility follows an overall downward trend until the rest of the data frame, with a few spikes ranging between 2% and 4%.

To measure the count stability, the standard deviation was adopted as the dispersion measure to compare both distributions. Concerning the iShares Core S&P 500 ETF data frame, the standard deviation of the sampled Dollar and time bar counts were calculated and were respectively $\sigma = 413.39$ and $\sigma = 218.97$. These values indicate that the time bars have more stable counts than Dollar ones and are closer to the 77263-valued mean. Regarding the second data frame, the standard deviation of the previously mentioned bars was $\sigma = 368.24$ and $\sigma = 271.97$. Similarly, time bars had the most stable counts. Lastly, Dollar and time bars in ATOM/USDT cryptocurrency were respectively 6826.89 and 1375.11. Time bars are expected to be more stable than the Dollar type since the first type is sampled in a constant frequency.

5. Experimental Setup and Methodology

This section describes the experimental setup adopted in this work. Trading is formulated as a Markov decision process (MDP), describing the state, reward, and the agent's architecture. Finally, the back-testing process and chosen hyperparameters values are also discussed.

5.1. Trading formulated as a Markov decision process

Markov decision process is a time-discrete stochastic control framework that provides a mathematical foundation to model decision-making problems. In MDPs, a system is in some given state and moves forward to another state based on the agent's decisions (decision maker). These interact continuously.

The environment is formulated as a set of states with which the agent can interact, choosing between buy, hold, and sell actions. The agent selects actions, and the environment responds to them by presenting new situations to the agent, as well as rewards which consist of numerical values that the decision maker seeks to maximize over time through its actions. The agent's goal is to control the system to maximize some performance criterion, in this case, profits.

5.1.1. Trading environment

Figure 10 shows the schematic of the proposed MDP to model an asset trading market. The environment is defined as the entity comprising everything outside the agent. More specifically, the agent and environment interact at each of a sequence of discrete time steps $t = 1, 2, 3, \dots$. At each time step t , the agent receives some representation of the environment's state $s_t \in S$ where S is the set of all possible states. The adopted environment sets each time step representing every time the market value reaches the predefined value.

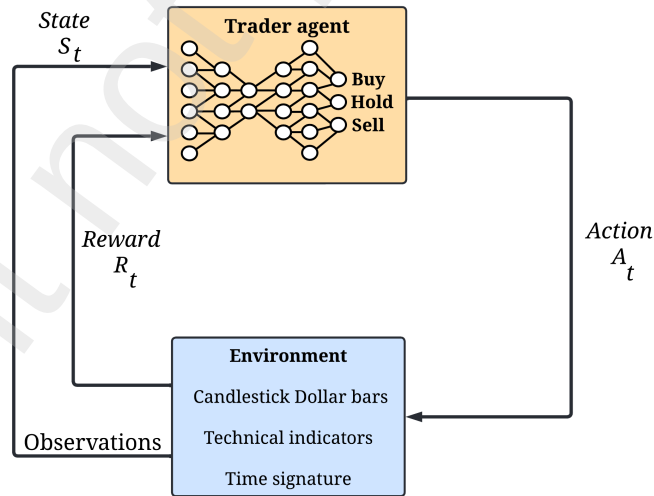


Figure 10: Trading formulated as a Markov decision process, making observations to the environment, taking actions and receiving rewards at the end of a trade.

The state of a system provides information that helps the agent to take action; hence this is the input. In order to predict the action better than random, the features in the state must be at least weakly predictable and stationary (have a stable mean). The market environment used the following input features:

- Candlestick Dollar bars
- Time signature
- Technical indicators

To construct the state, tick data, which is the record of transactions and prices of an asset, is grouped until it reaches a pre-defined market value, and a **Dollar bar** is generated. In the following, the current date time from the generated bar is collected, and outliers are removed according to the IQR method, considering the last seven days according to the date. Once the data is cleaned, the state can be assembled. According to the current date time, the state searches for the previous *LOOKBACK* amount of data samples of every feature and appends them. Every feature is normalized to prevent a large-valued variable from dominating the entire neural network.

Time signature features, such as the current time of the day and week, were also included in the state. These features may have a significant impact on profitability. Adding a particular time of the week may provide additional information that may not be apparent from a conventional numerical timestamp. These events could occur in regular weekly intervals, such as Federal Open Market Committee Meetings. Aiming to calculate the current time of the day, the hour and minute corresponding to the current index position are retrieved and multiplied by 24×60 to map that into float number, and then normalization is applied. In order to calculate the day of the week, the corresponding day of the current index position is retrieved and transformed into an integer number divided by 6 to normalize it.

Technical indicators were also included to construct the state vector. The relative Strength Index was used within 14 days basis on the closing prices. It is a Momentum Oscillator used to measure the strength or weakness of the asset. In addition to it, the state included the Balance of Power. It determines the pressure on the buyers and sellers by looking at how strongly the price has changed. Aiming to identify trends, the Aroon Oscillator also served as input tracking the highs and lows for the last 25 periods. This oscillator measures the current trend's strength and provides information about its persistence. Lastly, the Moving Average Convergence/Divergence indicator was used for trading trends. It is calculated by subtracting the

value of a faster period Exponential Moving Average from a slower one. When this indicator crosses above zero is considered bullish, while crossing below zero is bearish.

5.1.2. Agent

The agent is the entity that makes decisions based on rewards and punishments, seeking to maximize the total reward in a particular environment. This work implemented the agent using two deep feed-forward artificial neural networks.

The first neural network, also mentioned as the Online Q-Network, is used for getting the probability mass function $p_{\theta_i}(s, a)$, given the current state s and action a , the best action a^* from the next state s' .

The second neural network, named target Q-network, generates the probability mass function given the next state s' , using the action caused by the Online Q-Network $p_{\theta_i}(s', a^*)$. Using the Bellman equation in the projected distribution of $p_{\theta_i}(s', a^*)$, a probability mass function is generated to calculate the target values for the current state s .

Since the two function approximators have seen different samples, it is not likely to overestimate the same action. Figure 11 shows both neural network inputs and outputs.

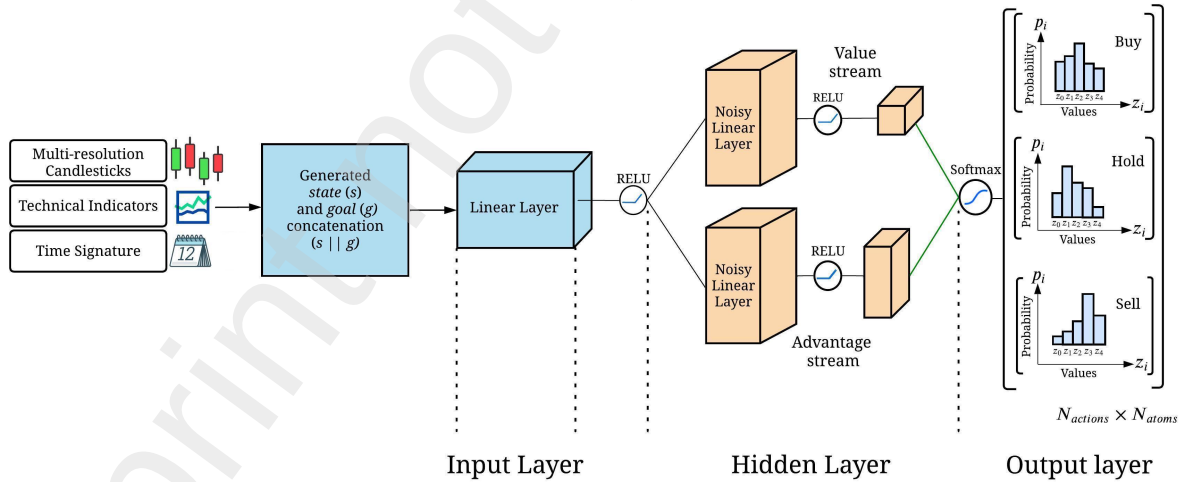


Figure 11: Adopted Deep Dueling Feed-Forward Q-Neural Network architecture in the trading problem. The architecture includes linear layers, noisy linear layers, RELU and Softmax activation functions. The input layer takes a state concatenated with a goal as input, the hidden layer splits into Value and Advantage streams, followed by the Softmax activation function. The output layer generates a probability mass function of each action.

Both deep neural networks' architectures are composed of input, hidden, and output layers. The input layer size corresponds to the size of the state s concatenated with goal g . It comprises a standard linear layer, performing linear operations to the input data. Next, it is followed by a RELU.

The hidden layer adopts the Dueling network architecture, consisting of two noisy linear stream layers that each provide separate estimates of the value and advantage functions. The value stream outputs the state value function $V_{\pi}(s, g)$ with the dimension of N_{atoms} , which are the "canonical returns" of the distribution.

The advantage stream outputs the advantage of the actions in the state s with dimensions $N_{actions} \times N_{atoms}$. The Noisy linear layers substitute the standard ϵ -greedy into the choice between exploration and exploitation. It injects factorized Gaussian noise to perturb the network's weights, leading to better exploration. Lastly, advantage and value streams are combined and followed by the Softmax activation function, which transforms values into probabilities.

The output layer consists of the array of 3 probability mass functions, one for each action buy, sell, and hold. Each probability mass function shows the probability that each return would have to be chosen in a certain action. The output layer has a dimension of $N_{action} \times N_{atoms}$.

5.1.3. Reward

Exponential profit and loss were adopted as the reward function. It is calculated by subtracting the current price (p_t), taking into consideration the transaction cost t_c involved when exiting the position, subtracted by the entry price (p_0), also taking in consideration t_c in respect to starting the trade. All variables previously mentioned are divided by the entry price (p_0), also taking into consideration t_c and multiplied by the respective *position* taken in the trade (1 for long and -1 for short). Finally, the natural exponent is taken of the respective value. Equation 22 shows the reward function:

$$R((s || g), a) = e^{\frac{p_t(1-t_c)-p_0(1+t_c)}{p_0(1+t_c)} \times position} \quad (22)$$

The Exponential profit and loss were chosen as the reward function based on the assumption that the agent should receive a greater reward for greater returns. At each time step, the agent will predict whether the trade is over or not. The first option will consider the current position, and the agent will provide the reward. On the other hand, if the trade has yet to be over, the agent will take action and will not receive any reward.

5.2. Backtesting Process

This subsection describes the backtesting process of training the agent to identify trading opportunities. The proposed algorithm contains a couple of steps: the **sampling** and **learning** phases.

The sampling phase consists in integrating with the environment, collecting experience tuples, and sampling them to the learning phase according to priority. On the other hand, the learning phases consist in feeding the batch of the sampled experience tuples and calculating the target and the loss function. Lastly, the algorithm performs the backward pass to transmit what it has learned from the loss function to its neural network weights. This way, the algorithm can approximate a function that maximizes the goals defined by the designer. Figure 12 shows an overview of the described process of backtesting implementation.

The application starts by instantiating the market environment, the main and target Q-artificial neural networks, and the hindsight-prioritized experience replay buffer. A new episode starts, and a state is generated for each time step. It is a combination of market features observed at a certain time-step t presented to the agent.

The state combines multi-resolution Dollar bars, technical indicators, and time signature features. The first feature is regarding the asset's price and is sampled every moment the market value reaches a predefined threshold. Technical indicators were also calculated from the price feature and concatenated to the state. Lastly, the current day of the week is served as time signature features to capture an event that may occur in weekly intervals.

In the following, a goal is generated according to a normal distribution. It is concatenated with the state, composing the final tensor ($s \parallel g$) that serves as input to the Online Q-Network. Since this is the start of the algorithm, it can't get the terminal state and put it as the goal; hence it is randomly generated. Then, the tensor ($s \parallel g$) serves as input to the Online Q-Network and takes action a according to the policy π .

Once the environment receives action a , it generates a next state s' , an immediate reward r , and whether the trade is over. The algorithm concatenates the chosen goal g with the following state s' forming the tensor ($s' \parallel g$) and stores it as an experience tuple in the hindsight-prioritized experience replay. This experience tuple contains the current state s the agent is in concatenated with the chosen goal g ($s \parallel g$), the taken action a , the immediate reward received r , the next state s' received concatenated with the goal g ($s' \parallel g$) and if the trade is over or not.

In the following, if the designer wishes to set additional goals to the hindsight-prioritized experience replay to create a broader perception of subgoals to the agent, a new goal g' is selected according to a predefined rule and concatenated with the current state s ($s \parallel g'$) and next state ($s' \parallel g'$) and stored as experience tuple in the replay buffer. The accumulated experiences at their initial moment have a maximum priority of 1.0 since they should be sampled at least once by the replay buffer.

Prioritizing experiences introduces bias due to the sampling proportion corresponding to the temporal difference error. The algorithm uses importance-sampling weights (Equation 11) that fully compensate for the non-uniform probabilities $P(i)$ if $\beta = 1$ to tackle this problem. The hyperparameters β starts at 0.4 and is linearly increased during every time step and reaches 1.0 at the end of training.

The learning phase only begins when the experience replay contains a number of tuples greater than the defined batch size to train the neural network. If it does not, the algorithm repeats the same previous steps until it reaches enough experience. On the other hand, if the memory size is greater than the batch size, the algorithm gets experience tuples indexes according to their priority using Equation (10). It is a stochastic sampling method that alternates between pure greedy prioritization and uniform random sampling.

In addition to it, the probability weights $P(i)$ are corrected and computed using 11 to compensate for the non-uniform probabilities $P(i)$ saving the correct weights in the w_i array. In the following, the learning phase starts. This phase consists in calculating the target for the distributions generated by the network and consequently its loss ($L(\theta_i)$). This phase will be covered in further detail in the following subsection. After calculating $L(\theta_i)$, an element-wise multiplication is performed between w_i and $L(\theta_i)$, and the neural network performs the backward pass to adjust its weights.

The learned experience's priorities are updated by summing the proportion of their temporal difference error with a minimum value of ϵ to make priorities > 0 . Lastly, if the trade did not finish, the algorithm repeats all the previously described steps until it is. Once the trade is liquidated, the target Q-network performs a hard copy update from the Online Q-Network weights.

Figure 13 shows the overall learning phase that includes taking the sampled batches generated from the sampling phase, calculate the target distribution and the loss of the neural network.

The learning process of the algorithm starts by sampling a batch of tuples based on priority from the replay buffer. The batch size was 1 for illustration purposes. In the following, the next state concatenated

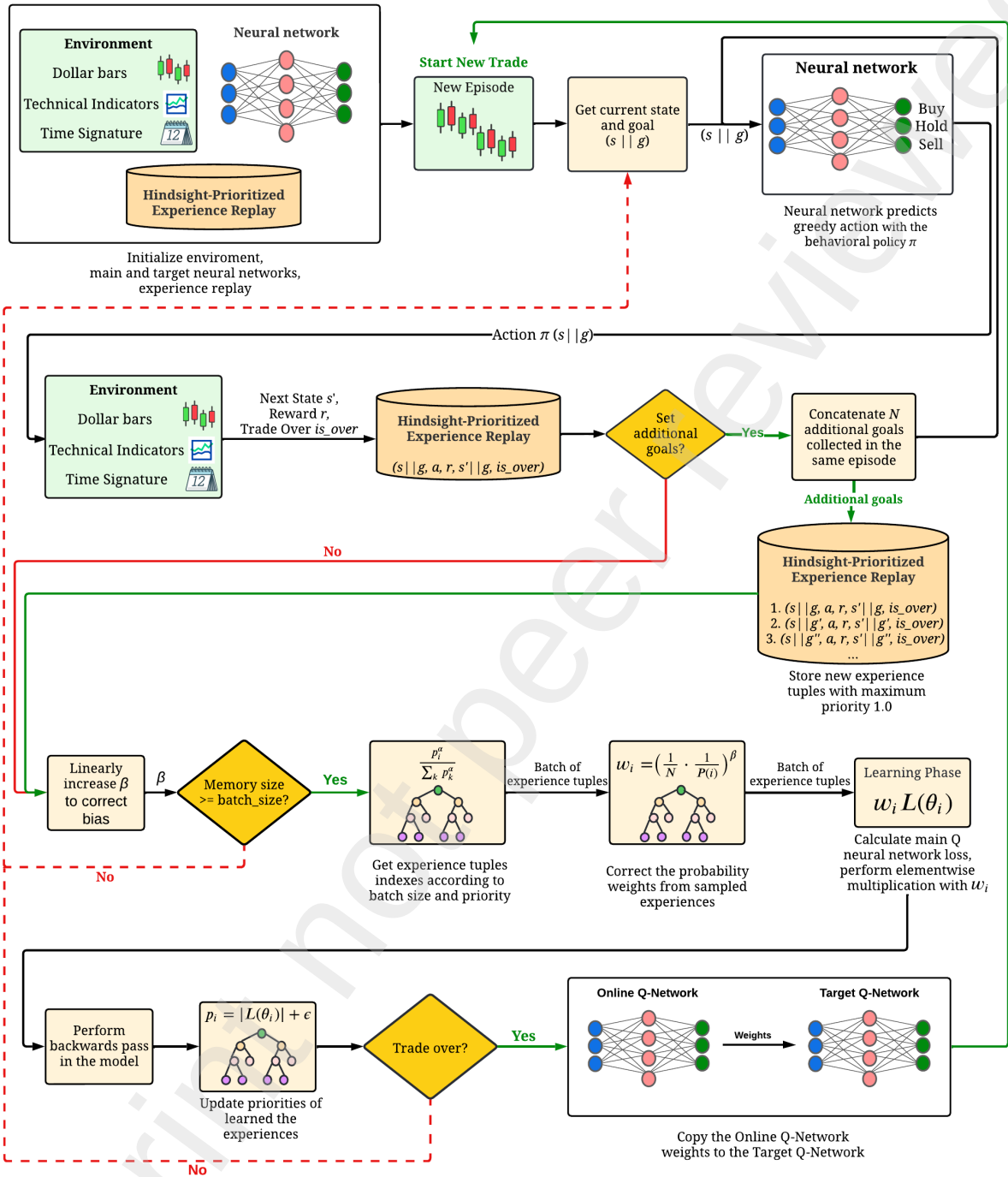


Figure 12: Backtesting overview of the ETDQN strategy, consisting of resource initialization, state assembly by combining Dollar bars, technical indicators, and time signature features, followed by the objective concatenation. In the following, the tuple of experience is created and stored. Finally, priorities are updated, and target network weights' are updated at the trade liquidation.

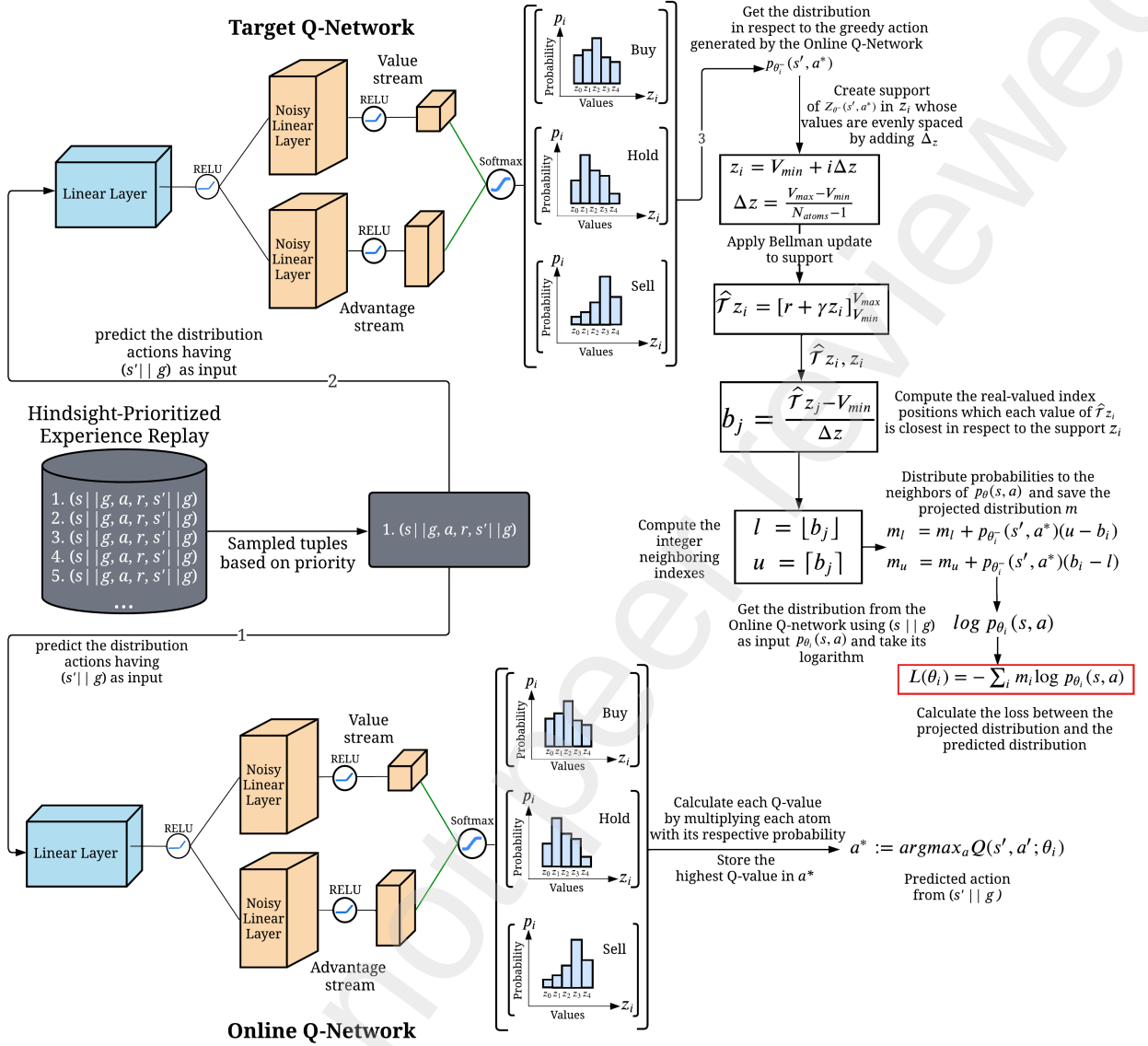


Figure 13: ETDQN learning process consists of storing the best action from Online Q-network, letting the Target Q-Network evaluate it, and get its respective distribution. In the following, a discrete-valued support is created, and Bellman update is applied to it. Then, probabilities are distributed to the distribution's closest neighbors, shifting the predicted distribution. Finally, the cross-entropy loss between the actual value distribution $p_{\theta_i}(s, a)$ and the shifted distribution m is calculated, performing back-propagation in the Online Q-network.

with the goal $(s' || g)$ serves as input to the Online Q-Network. The first linear input layer of the network processes $s' || g$ and gets split into two branches: the value and advantage streams.

This division benefits learning since the network learns to estimate the distribution of values only for important conditions; hence it learns what states and goals are more valuable. It is important in sparse-reward environments since feedback only occurs occasionally. Each branch is composed of noisy linear

layers, which add parametric noise to the network weights used to drive exploration. The layer contains a hyperparameters called σ , the equivalent initial standard deviation for creating the perturbation. The higher this hyperparameters is, the more energy the injected noise perturbs the weights. The value and advantage streams are combined again and followed by a softmax layer to produce the final probability distributions, one for each action.

Then, for each distribution, each atom is multiplied with its respective probabilities to generate Q-values, one for each distribution. The greedy action a^* is the distribution that returned the highest Q-value. This action is stored, and the same procedure of predicting the probabilities of each action from $(s' || g)$ occurs in the target network.

However, the target network only evaluates the action. Instead of generating Q-values and selecting the greedy action, the greedy action generated by the online Q-Network stored is retrieved, and its probability distribution is collected in the target network.

This probability distribution is denominated $p_{\theta-}(s', a^*)$. To perform the Bellman update, a support z_i array is created. It contains the same number of atoms that the $p_{\theta-}(s', a^*)$ distribution. However, its values are evenly spaced, ranging from minimum V_{min} and maximum V_{max} values.

In the following, the Bellman update $\hat{T}z_j$ is performed concerning the support z_i . z_i is multiplied by the discount factor γ and summed to the immediate reward r . Since adding r dislocates the distribution in the x-axis, the update may be beyond the set $[V_{min}, V_{max}]$ intervals.

To tackle this issue, $\hat{T}z_j$ is clipped into the previous range. To project $\hat{T}z_j$ closest to the support z_i , the b_i array computes the closest real-valued index positions, which each value of $\hat{T}z_j$ is closer in respect to z_i and from that its integer lower and upper neighboring indexes.

The probabilities of $p_{\theta-}(s', a^*)$ are distributed and saved to the projected distribution m . In the following, the Online Q-Network predicts the distributions from the current state concatenated with the goal tensor $(s||g)$, and the distribution corresponding to the action a in the experience serves as the chosen action. This distribution is denominated $p_{\theta}(s, a)$.

Finally, the loss between the project distribution m and $p_{\theta}(s, a)$ is calculated concerning the cross-entropy loss function, which measures the difference between two probability distributions.

5.3. Hyperparameters

This subsection describes the set of hyperparameters necessary to tune the model. Table 3 shows the chosen values for each hyperparameters.

- **Learning Rate:** Determines the step size at each iteration while moving toward a minimum of a loss function.
- **Loss Optimizer:** Optimizer algorithm to reduce loss.
- **Loss Function:** Maps the distance between the current output of the algorithm and the expected output.
- **Activation Function:** Function that adds non-linearity to the neural network
- **Number of Actions:** Number of discrete actions that agent can take. Buy, sell, and hold.
- **Lookback:** Number of previous timesteps that are used in order to predict the subsequent timestep.
- **Batch Size:** Number of samples that are propagated through the neural networks.
- **Discount Rate:** Reflects both time value and risk compensation.
- **Memory Size:** Maximum amount of tuples that the replay buffer can store.
- **Additional Goals:** Number of Goals to add to the Hindsight Experience Replay
- **Noisy layer initial standard deviation (σ_0):** Coming from Factorized Gaussian linear layer. Initial standard deviation to inject noisy into the neural network's weights.
- **Prioritization of experiences:** Amount of prioritization of experience tuples.
- **Distributional Atom (N_{atoms}):** Number of canonical values generated from the distribution.
- **Distributional minimum and maximum values**
(V_{min}/V_{max}): Maximum and minimum value of the generated distribution.

At the first attempt, the learning rate was set to the standard rate indicated for the Adam optimizer, which is 0.001. However, the algorithm presented a poor performance by not being able to converge to a good policy. Then, according to DRL similar papers, the adopted learning rate of 0.0000625 demonstrated

Table 3

Hyperparameters values used in ETDQN applied to iShares S&P500 ETF, Western Digital Corporation, and ATOM/USDT cryptocurrency pair assets.

Hyperparameter	Assets		
	iShares Cores S&P500 ETF	Western Digital Corporation	ATOMUSDT cryptocurrency
Learning Rate	0.0000625	0.0000625	0.0000425
Loss Optimizer	Adam	Adam	Adam
Loss Function	Cross-entropy	Cross-entropy	Cross-entropy
Activation Function	Rectified Linear Unit	Rectified Linear Unit	Rectified Linear Unit
Lookback	20	20	20
Batch Size	8	16	8
Discount Rate (γ)	0.99	0.99	0.99
Memory Size	500	500	600
Additional Goals	5	3	4
Initial Standard Deviation (σ_0)	0.5	0.8	0.3
Experiences prioritization (α)	0.5	0.5	0.5
Prioritization bias compensation (β)	0.4 $\rightarrow$ 1.0	0.4 $\rightarrow$ 1.0	0.4 $\rightarrow$ 1.0
Distributional atoms (N_{atoms})	51	51	51
Distributional min/max values (V_{min}/V_{max})	[-20, 100]	[-20, 100]	[-20, 100]
Market value sampling threshold	500K USD	1M USD	7.5K USD

an improvement. Initially, this rate was adopted to optimize the stochastic gradient descent, but it showed promising results in Adam. In the network architecture, RELUs were adopted as activation functions except for the last layer, which contains the Softmax function. RELUs were chosen instead of other types because they avoid nonlinear activation functions and reduce complexity. Lookback was set to 20. This parameter was highly sensible to tune.

The model showed a higher variance during the training when the lookback parameter was greater than 60. The same fact occurred in the batch size. The higher both parameters, the more variance in the results. The opposite is also valid. The discount rate was set to 0.99 to make the agent seek to learn as long-term as possible. The memory size was set to 500. The memory size did not have to be expressive due to the prioritization feature implemented, allowing the agent to keep the most important transitions. To sample sub-goals to make the agent learn as much as possible from alternative outcomes, additional goals were fed to the agent. Each data frame had distinct amounts, having the iShares Cores S&P500 ETF the maximum amount of 5 different goals. To perform better exploration, the initial standard deviation set to perturb the noise of the neural network was 0.5, 0.8, and 0.3, respectively.

In most volatile assets, exploring much is not a good idea due to the possibility of a loss. The more volatile the asset, the lower σ_0 . The prioritization exponent, which sets how much prioritization to set in transitions, was set to 0.5. The value was set according to the original implemented paper. The same applies to the β parameter that compensates for the non-uniform distribution sampling which is linearly increased at every time step from 0.4 to 1.0. This hyperparameters was set to 0.6. Lastly, the distribution's number of

"canonical returns" was 51 as the original implementation of the categorical DQN algorithm. However, the minimum and maximum values that the distribution can have were set to -20 and 100. Some other values such as $[-10, 10]$ and $[0, 200]$ were tried. However, the best results were achieved with $[-20 \text{ and } 100]$ the pair.

6. Results

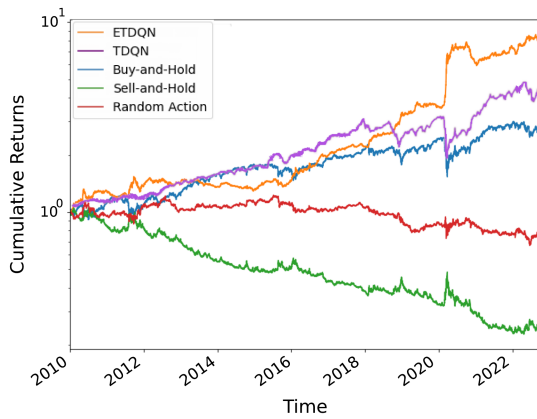
This section presents the results of the conducted experiments. In order to evaluate and compare performance, daily cumulative returns, 6-month Sharpe ratio, maximum drawdown, monthly and annual returns were calculated for every benchmark and ETDQN, followed by an analysis contemplating each of them. Transaction fees (t_c) were taken into consideration. The adopted t_c in ATOM/USDT cryptocurrency was 0.03% and 0.18% for the remaining.

6.1. Cumulative returns

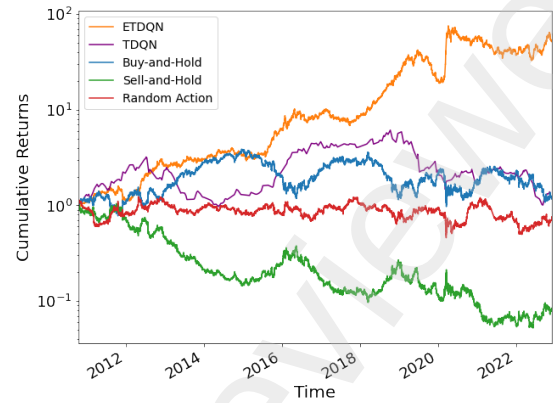
The cumulative returns metrics show the historical evolution of the portfolio value and not only its final evaluation. Figure 14a shows the daily cumulative returns regarding the abovementioned strategies applied to the iShares S&P500 ETF, Western Digital Corporation, and ATOM/USDT data frames.

As indicated by Figure 14a, the ETDQN agent outperformed all benchmarks regarding cumulative returns concerning the minimum, mean, and maximum value in iShares S&P500 ETF. The strategy had a mean value of 3.86, reaching a maximum of 8.61 and a minimum of 1.0 between 2010 and 2022. The policy did not lose the invested amount at any moment during the 12 years. Starting in 2010, the model had a good performance, outperforming every benchmark. It formed a local peak in April and struggled to learn a good policy until 2016. According to Figure 7d, a 6% volatility spike occurs, and the agent takes advantage of it, overtaking the lead in performance. The strategy keeps increasing the profits compared to the remaining onwards. In March 2020, the model achieved its best performing moment during the COVID-19, having an outstanding 164% profit, reaching 8.61 cumulative returns. Followed by a -6.9% drop in April, the agent keeps increasing its profits and ends its portfolio evaluated at 7.60.

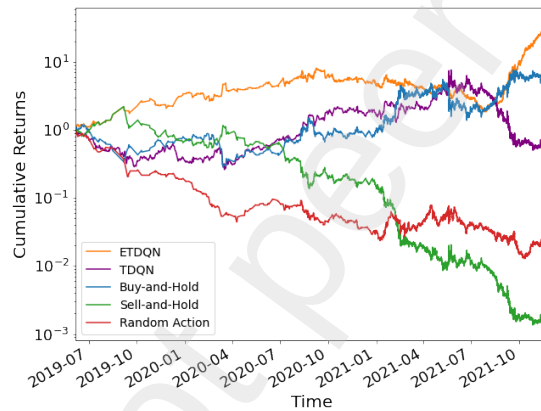
TDQN benchmark had a mean of 2.64, a maximum of 4.92, a minimum of 0.95. Even though the strategy is increasing between 2010 and 2013, still does not perform better than the previous strategy. However, in 2013 the TDQN takes the lead, following an up-trend reaching 2.63 cumulative returns. Unfortunately,



(a)



(b)



(c)

Figure 14: ETDQN daily cumulative returns between 2009 - 2022: (a) iShares S&P 500 ETF, (b) Western Digital Corporation, and (c) ATOM/USDT cryptocurrency pair.

differently from the agent strategy is not able to identify and take advantage from the 2019 and 2020 dumps. From 2021 onwards, the strategy cumulative returns keeps increasing achieving its maximum value.

The B&H benchmark had a mean of 2.06, a maximum of 3.01, and a minimum of 0.91 concerning daily cumulative returns. The strategy ranges between 1.0 to 1.43 until the end of 2011, when the asset starts an up-trend that remains active until 2016, Having the second position concerning performance. In the following, the strategy has a short-term downtrend having -2.2% and -5.3% drops, respectively, in December 2015 and January 2016. In addition, the approach formed a local bottom in December 2018 with a -10.1% dump, followed by a recovery process; the asset was affected by the COVID-19 during March 2020, having

its worst month with a -15.7% dump. The portfolio assumes an up-trend and ends the backtest evaluated at 2.61, recovering from the drop in a v-shaped way,

The S&H benchmark had a poor performance with a mean of 0.23 of the initial invested amount. This strategy assumes the worst position regarding performance. A maximum portfolio value of 1.05 was achieved, as well as a minimum of 0.23. Visually analyzing the strategy, a down-trend pattern in the long term is observable. Some positive spikes during 2012, 2019, and 2020 are detected, even though a reversion and continuation of the down-trend follow them.

Lastly, the Random Action strategy achieved the third position regarding performance with a mean of 0.96, a maximum of 1.19, and a minimum of 0.58. It achieved the third position regarding mean performance. During the 2010 - 2016 period, the strategy does not have a considerable variation in performance. In the following years, it assumes a slight down-trend finishing the backtest and having a 0.89 portfolio value.

Figure 14b shows the cumulative returns from the same strategies applied to the Western Digital Corporation from 2010 to 2022. Similarly to the previous data frame, ETDQN outperformed all benchmarks in the dataframe. In 2012, the strategy was evaluated at 0.93, achieving its worst period. From 2012 onwards, the policy stood out from the other benchmarks and doubled its initial investment. The strategy stabilized between 3.0 and 3.5 during 2014/mid-2015. At the end of 2015, it increased its value five times during a drop in the asset. In 2018, The policy increased its value by eight times, followed by a drop of 1.5. In the following, the model recovers its value during the COVID-19 and has its highest 164% pump in March 2019. After the event, the policy assumes a slightly sloped downtrend and finishes the backtest assuming an up-trend evaluated at 51.94. In this strategy, the mean portfolio value was evaluated at 17.12, meaning that the investment would have increased to the described amount.

TDQN had a minimum value of 0.87, a maximum of 6.12, a mean of 2.40. The strategy had the second best position regarding performance. Between 2010 and 2013, this strategy is able to outperform the previous policy, achieving a peak of 3.1. However, from 2013 onwards the strategy is not able to generalize well staying on second position until the end of the backtest. Between the years of 2013 to the half-2014, the strategy's follows a downtrend, followed by a recovery in 2015 reaching its highest 6.12 evaluation in the end of 2018. Unfortunately, from 2019 onwards, the fails on taking advantage of the COVID-19, reaching its 0.87 lowest value on 2022.

With respect to the B&H benchmark, the strategy had a minimum evaluation of 0.78 during 2012, a maximum of 3.86 in 2015, and a mean of 2.13, achieving second place regarding performance. The policy oscillated between 1.0 and 1.5 from 2010 until 2014. An uptrend started and went up to 2015 when the asset had its best-evaluated value of 3.86. In the following, the strategy loses all profits earned from the previous trend and goes back to 1.2 in the most significant dump of the asset of -78% to its maximum value. The same pattern repeats from 2016 to 2019, but with a lower margin. The asset had a -58% dump during the COVID-19 event, followed by a recovery in 2021. The asset finishes the backtest evaluated in 1.21.

S&H had achieved the fifth and last position regarding performance against the remaining benchmarks. The policy's evaluation is a minimum value of 0.05 in 2022, a maximum of 1.06 in 2010, and a mean of 0.23. During the 2010-2015 interval, as opposed to the previous strategy, a strong downtrend was predominant until the middle of 2015. From 2015-2016, the strategy has one of its best performance moments recovering its portfolio with an 11.2% pump. Followed by the asset value increase in 2016, the S&H benchmark policy is penalized again, reverting its profits and dropping -21.2% while it traded in 2018 with a 28% profit. The strategy keeps decreasing its value until reaching 0.05 of the initial investment.

Lastly, the Random Action strategy had the fourth place with a minimum value of 0.45. The strategy assumed a downtrend from 2010 to 2019. It assumes an uptrend during the next three years, followed by another downtrend.

Figure 14c shows the cumulative returns from the same strategies applied to the ATOM/USDT from 2019 to 2022. ETDQN outperformed all benchmarks in the data frame. The evaluated policy metrics were a minimum value of 0.95 in August 2019 and a maximum value of 37.62 in November 2021. With a portfolio value performance mean of 12.08, the strategy starts with fewer bars than the remaining datasets and, throughout the backtest course, has its amount increased. During September 2019 and July 2020, the strategy increased its initial investment by three and then decreased performance. During this period, B&H significantly increases quickly and gets equivalent to the agents' performance. Fortunately, in July 2021, the agent is smart enough to capture the asset's upward trend and outperforms the B&H benchmark by opening short positions during the price fall.

TDQN had a minimum of 0.26, a maximum value of 6.12, and a mean of 2.40 cumulative returns. Between the intervals from 2019-07 to 2020-03, the agent struggles to be competitive, reaching its lowest value of 0.26 in March 2020. Fortunately, the policy is able to learn from the experiences and follows an

up-trend from April 2020 onwards outperforming the B&H strategy in the end of June 2020 and S&H benchmark in July. The strategy reaches its maximum peak value of 6.12 during May 2021, surpassing even the previous strategy at the moment. Unfortunately, differently from the previously mentioned policy, the agent is not able to generalize well at the moment and has a worse performance than B&H benchmark, assuming a downtrend until the end of the backtest.

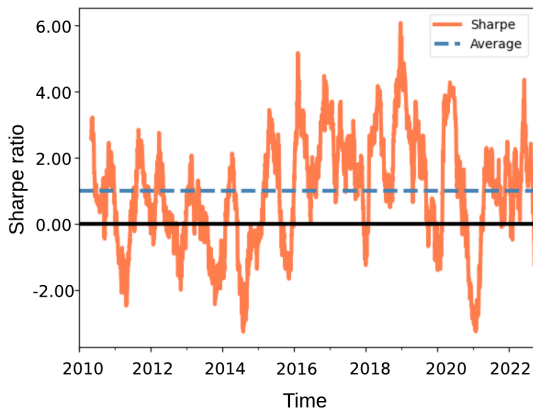
The B&H achieved second place in performance, being evaluated at 0.29 during its minimum value in August 2019 and at 7.67 during its maximum performance in August 2021. The strategy had a 5.65 cumulative return mean regarding performance. Between July 2019 and April 2020, the strategy had its worst performance having its minimum evaluated performance value within this interval. In contrast to this period, the cryptocurrency market's Bullrun starts, and the asset multiplies its value by 8, recovering its losses and generating profits. From April 2021 onwards, the strategy moves sideways, followed by a -33.9% drop during May 2021. The strategy forms a local bottom followed by another 52% pump between July and October, stabilizing its value at 6.84.

The S&H strategy had the fourth position regarding cumulative returns contrasting the previous strategy. In addition, the policy had its worst performance equivalent to its mean of 0.01. If the investor tried to allocate all capital into a short position, the entire investment would be lost. During the start of the backtest, the strategy had its best performance moment of 2.22, outperforming even the ETDQN. However, the strategy assumes a downtrend and does not recover from it.

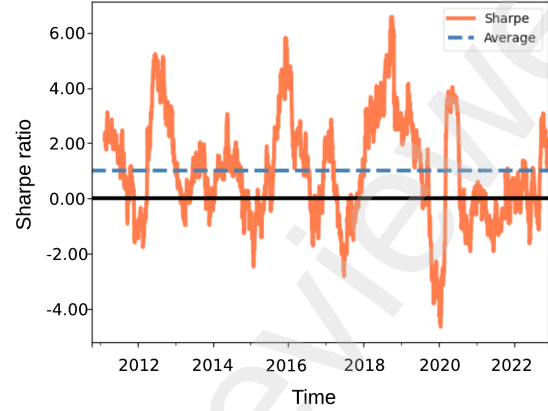
Lastly, the Random Action strategy achieved third place regarding performance. This policy achieved its worst moment of 0.01 accumulated returns in October 2021 and its best performance moment at the start of the backtest. This policy had a better position than the previous strategy due to its best performance between January and April of 2021, even though both assume downtrends during the entire backtest.

6.2. Sharpe ratio

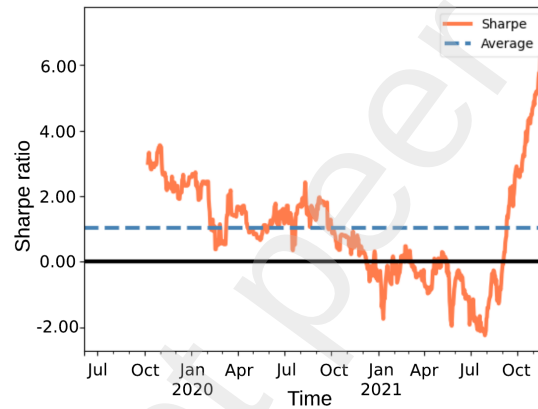
This subsection analyzes the Sharpe ratio metric for each data frame and benchmark. The metric adopted a 126-day window value to analyze the ratio on a 6-month rolling basis. The estimated number of days in a month was 21 since only business days were considered. Figure 15 presents the historical 6-month rolling Sharpe ratio generated by ETDQN applied to the iShares S&P500 ETF, Western Digital Corporation, and ATOM/USDT data frames.



(a)



(b)



(c)

Figure 15: ETDQN 6-month rolling Sharpe ratio between 2009 - 2022: (a) iShares S&P 500 ETF, (b) Western Digital Corporation, and (c) ATOM/USDT cryptocurrency pair.

ETDQN was evaluated during its worst-performing moment with a minimum Sharpe ratio of -3.26 in 2021. The agent also achieved a 6.07 ratio in 2019 during its best performance and a mean of 1.04. At backtest start, the policy oscillates its Sharpe ratio between the -2.5 / 2.5 range, followed by a breakout in 2015. In the following, the strategy achieves a 5.2 ratio performing an interesting risky-reward relation. In 2019, the strategy kept increasing the ratio and achieved its best relation with 6.07 by taking advantage of the COVID-19 event dump and opening a short position. Unfortunately, this period is followed by the worst performance moment in 2021, achieving a -3.26 ratio. Luckily, the strategy recovers from this loss in a v-shape way and reverts to a mean value. This policy has the first position regarding the mean metric against the benchmarks.

TDQN was evaluated with a minimum of -2.66, a mean of 1.03 and a maximum of 4.56 6-month rolling Sharpe ratio. Its best risk-reward moment occurs in 2013, going from -1.24 evaluation to 4.56. During this period this strategy outperforms all of remaining. In addition, its worst risk-reward moment happens in 2020 during the start of COVID-19 pandemic. During this period, the Sharpe ratio has a dump from 3.62 to -2.66. The strategy achieves the second place regarding risk-opportunity relation according to the mean.

B&H was evaluated with a minimum value of -2.97 ratio during its worst risk-reward moment in 2019 and its best moment with a 4.6 ratio in 2017. The strategy's Sharpe ratio oscillates between 3.0 and -2.5, with few exceptions during 2010 and 2016, and has a 0.76 mean. The policy achieves its best risk-return measure in the following year, even though a 150% dump occurs subsequently. This policy achieved third place regarding risk-opportunity relation according to the mean.

S&H achieved the last position regarding risk-return relations, reaching a -4.6 Sharpe ratio during its worst moment in 2018 and 2.97 in 2019 during the COVID-19 event. It is also worth mentioning that since this strategy is the reverse of the previously analyzed one, it achieves its best moment of risk-return relation once the previous strategy reaches its worst scenario.

Finally, the Random Action strategy achieved the fourth position regarding risk-return relation. This policy had its worst moment Sharpe ratio evaluated at -2.95 in 2021, its best rolling ratio estimated at 2.81 in the same year, and a 0.0 mean.

Figure 15b shows the 6-month rolling Sharpe ratio calculated in the Western Digital Corporation data frame regarding all benchmarks. ETDQN had achieved the best position compared to the benchmarks regarding the mean Sharpe ratio of 1.0. In addition, according 15b, during 2020, the policy had its worst ratio of -4.64, followed by a recovery in a v-shaped way. Its best risk-return relation was achieved during 2018 when the model assumed a strong uptrend. As expected at the beginning of the backtest, the agent struggled to learn an adequate risk-return relation when in 2010, its ratio had a 200% pump. The ratio oscillates between 6.0 and -2.0, except during 2020 when its highest drop occurs from its maximum value of 6.57 to its lowest; hence, this is the strategy's most volatile Sharpe ratio moment.

TDQN had the third position regarding the Sharpe ratio. Although this policy and the B&H benchmark achieved the same 0.3 mean value, the maximum Sharpe ratio value was used as tie-breaking criteria; In addition, the strategy had a minimum of -3.43 and maximum of 2.76 6-month rolling Sharpe ratio. This policy has its worst risk-reward moment during March 2014 having a dump from 1.38 Sharpe ratio to -3.43.

On the other hand, its best risk-reward moment occurred during July 2016. The ratio had a v-shaped pump from -2.13 to 2.76.

B&H had the second position regarding the Sharpe ratio evaluation with a 0.3 mean. In 2019, the policy had the worst-performing moment when the Sharpe ratio dropped from 1.5 to its lowest value of -4.55. During the beginning of the backtest, the strategy achieved its best ratio of 3.51, followed by an -112% drop. The moments with the highest variation of the metric were during 2016 and 2019, with both of them recovering from it in a v-shape way.

S&H had its best and worst evaluated Sharpe ratios as opposed to B&H. Evaluated in a mean of -0.3 Sharpe ratio, the strategy occupied the last position regarding risk-return relation, even though the Random Action strategy was a mean evaluated in 0.13. Its best 4.02-performing moment evaluation was during the COVID-19 event in 2019. On the other hand, the policy suffered from the asset price recovery from the episode and had its worst risk-return rate of -3.62. Lastly, the Random Action strategy was evaluated with a mean Sharpe Ratio of 0.13 and occupied the fourth place regarding risk-return.

To summarize all information regarding the 6-month rolling Sharpe ratio, Table 4 presents the minimum, mean and maximum rolling Sharpe ratio values from ETDQN, TDQN, B&H, S&H benchmark, and Random Action strategies in respect to the iShares S&P 500 ETF, Western Digital Corporation and Binance spot ATOMUSDT cryptocurrency data frames.

It is possible to assert that the ETDQN had 1.36, 1.01, 2.74, and 1.4 greater mean Sharpe ratios having TDQN, B&H, S&H, and Random Action benchmarks as basis in iShares S&P500 ETF. The policy achieved the first position regarding this metric. In addition, the same policy had 3.3, 3.3, 6.6, and 7.7 better risk-return relations regarding Western Digital Corporation.

In contrast to the previous data frames, the strategy generated by the agent was not in the top position regarding the mean Sharpe ratio in ATOM/USDT, even though its maximum value was more significant than any of the benchmarks. This fact may be due to insufficient data generated at the beginning of the data frame and the asset's volatility.

6.3. Maximum Drawdown

This subsection presents the maximum drawdown metric for ETDQN, TDQN, B&H, S&H, and Random Action strategies, applied to the iShares S&P500 ETF, Western Digital Corporation, and ATOM/USDT data frames.

Table 4

Strategies' 6-month rolling Sharpe ratio metric, consisting of minimum, mean, and maximum historical value, applied to iShares SP500 ETF, Western Digital Corporation, and ATOM/USDT cryptocurrency pair assets.

	Strategy	Rolling Sharpe Ratio		
		Minimum	Mean	Maximum
iShares S&P500 ETF	ETDQN	-3.26	1.04	6.07
	TDQN	-2.66	1.03	4.56
	B&H	-2.97	0.76	4.6
	S&H	-4.6	-0.76	2.97
	Random	-2.95	0.0	2.81
Western Digital Corporation	ETDQN	-4.64	1.00	6.57
	TDQN	-3.43	0.3	2.76
	B&H	-4.55	0.3	3.51
	S&H	-3.51	-0.3	4.55
	Random	-3.62	0.13	4.02
ATOMUSDT Cryptocurrency	ETDQN	-2.24	1.02	7.3
	TDQN	-3.18	0.86	3.88
	B&H	-1.55	1.19	3.65
	S&H	-3.65	-1.19	1.55
	Random	-4.66	-0.82	2.44

Table 5

Maximum Drawdown in percentage generated by ETDQN, TDQN, B&H, S&H, and Random strategies between 2009 to 2022, applied to iShares S&P500 ETF, Western Digital Corporation, and ATOM/USDT cryptocurrency pair assets.

Metric	Strategy	Drawdown (%)		
		iShares SP500 ETF	Western Digital Corporation	ATOM/USDT Cryptocurrency
Maximum	ETDQN	-24.40	-56.60	-76.70
	TDQN	-35.1	-90.9	-83.5
	B&H	-56.60	-74.50	-74.0
	S&H	-78.00	-95	-99.9
	Random	-47.5	-65.40	-94

ETDQN had the lowest maximum drawdown in iShares S&P500 ETF compared to the remaining benchmarks. The algorithm had 10.7%, 32.2%, 53.6%, and 23.1% less risk of ruin than TDQN, B&H, S&H, and Random action benchmarks. The strategy's maximum drawdown of -24.40% occurred in 2021. In addition, TDQN and B&H had their maximum drawdown of -35.1% and -56.60 during March 2020, respectively. Both strategies were not able to capture this decline. This dump is related to COVID-19.

Concerning the Western Digital Corporation, ETDQN again had the lowest maximum drawdown compared to the benchmarks. The strategy had 34.3%, 16.4%, 95%, and 25.5% less risk of ruin than TDQN, B&H, S&H, and Random action benchmarks. ETDQN and B&H had their maximum drawdown of -56.60% during the COVID-19 event, contrary to TDQN, which had its worst drop in 2022.

Unfortunately, B&H had the lowest maximum drawdown in ATOM/USDT compared to the remaining strategies. However, ETDQN presented a very close maximum drawdown with a -2.6% difference. ATOM/USDT cryptocurrency pair had the most significant drawdowns since it was the most volatile asset. ETDQN had a maximum decline of expressively -76.70% in July 2021. In addition, the B&H benchmark also had an expressive -74.00% maximum decline during April 2020, during COVID-19.

6.4. Monthly Returns

This subsection analyzes the generated monthly returns by each benchmark to show the reader how much the strategy would profit monthly. Figure 16 shows monthly returns heatmaps generated by ETDQN in iShares S&P500 ETF, Western Digital Corporation, and ATOM/USDT data frames.

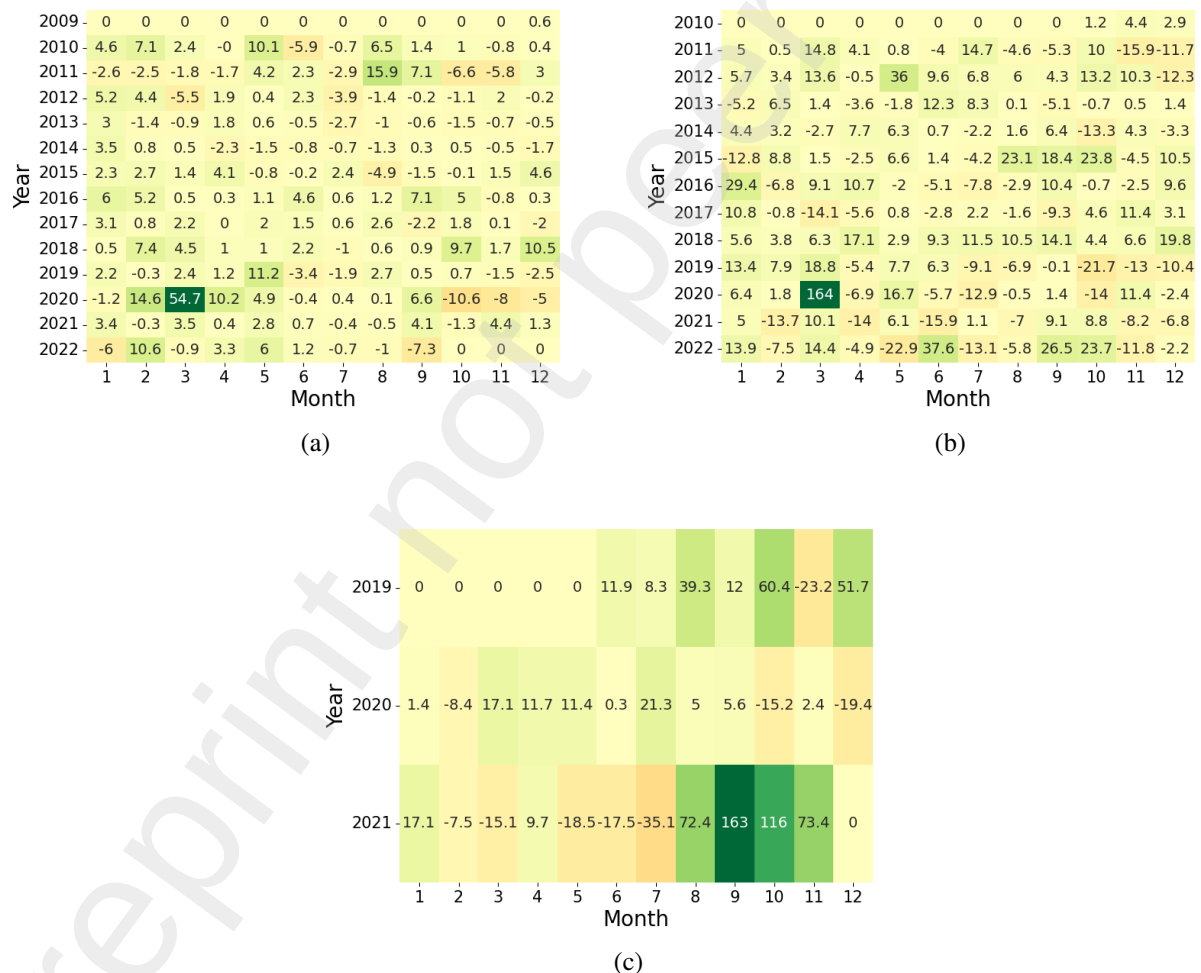


Figure 16: ETDQN monthly returns in percentage between 2009 - 2022: (a) iShares S&P 500 ETF, (b) Western Digital Corporation, and (c) ATOM/USDT cryptocurrency pair.

As shown in Figure 16a, ETDQN did not profit much in 2009 due to the lack of data. In the following year, the strategy profit had eight months of positive returns having its best performance in March with a 10.1% return. In August 2011, meanwhile, the TDQN-based, B&H, S&H, and Random Action benchmarks had -0.6%, 6.9%, 5.9%, and 5.3% returns, ETDQN had 15.9%, followed by another 7.1% positive return. Unfortunately, the strategy delivers both negative returns in the next couple of months. From 2012 to 2017, the profits generated by the agent did not have a high amplitude ranging between -2.5% to 5%, hence having a slight positive asymmetry. A few exceptions are present on both sides, for example, the 7.1% profit during September 2016 and the -4.9% loss in August 2015.

In 2018, the ETDQN again outperformed all benchmarks concerning February, October, and December, generating 7.5%, 9.7%, and 10.5% returns. At the exact moment, TDQN generates -5.3%, -7.6%, -10.2% returns, S&H -5.3%, -5.3\$, and -10.1%, B&H 5.1%, 5.3%, and 10.6% and the Random Action -3.8%, 2.6%, and 5.1%, respectively. It is observable that a dump in the asset price must have occurred during this period. Even though B&H also performed well in these price dumps, it cannot maintain its earnings like ETDQN. In the following year, 2019, the agent-based strategy outperformed all benchmarks once again in March, having an 11.2% return. In 2020, the proposed algorithm achieved its best performance compared to all benchmarks taking advantage of the COVID-19 pandemic event price drop. The model can subsequently generate 14.6%, 54.7%, 10.2%, and 4.9% returns. During 2021, the model strategy had profits between 3.5 and 4.4%, but in 2022, the agent gets volatile again, having returns ranging between -7.3% to 10.6%.

Comparing monthly returns generated from every strategy in Western Digital, it is observable that the ETDQN monthly returns heat map contains more uniform colors. With a few exceptions, the returns are within a specific range. During 2010, the strategy only provided positive returns. In the following year, it is possible to visually positive returns, such as 5%, 14.8%, 4.1%, 14.7%, and 10%, respectively, generated in January, March, April, August, and October. Unfortunately, a few expressive -4.6%, -5.3%, -15.9%, and -11.7% negative returns are spotted in August, September, November, and December. It is also worth mentioning that the backtest is still in an early phase; hence the model weights are not fully adjusted. In 2012, a negative -12.3% return only is placed in December, and some expressive 13.6%, 36%, 9.6%, 13.2%, and 10.3% positive ones are generated during March, May, June, October, and November.

In the following year, 2013, returns are more neural. The policy generated positive 6.5%, 12.3%, and 8.3% returns during February, June, and July, as well as some -5.2%, 5.1% declines in January and September,

but still leaving the strategy in a positive balance. During 2014, the most expressive return generated was -13.3% in October, even though some 7.7% and 6.3% positive ones during April and March counterbalanced the equation and ended up closing the year in a positive balance. Succeeding the previous year with a good margin regarding positive returns, 2015 resulted in a -12.8% negative return in January. However, in February, May, August, September, October, and December, expressive 8.8%, 6.6%, 23.1%, 18.4%, 23.8%, and 10.5% positive returns were generated by the policy.

During 2016, the strategy generated during January, March, April, September, and December expressive 29.4%, 9.1%, 10.7%, 10.4%, and 9.6% positive returns during 2016, even though -6.8% and -7.8% negative returns are spotted. Negative returns predominated the following year, 2017. Expressive returns such as -10.8%, -14.1%, and -9.3% were generated during January, March, and September, and only 11.4% expressive positive returns were generated.

Outperforming the previous year, 2018 generated the expressive 17.1%, 9.3%, 11.5%, 10.5%, 14.1%, and 19.8% predominating positive returns. During 2019, the positive/negative return ratio is balanced with expressive positive returns such as 13.4%, 7.9%, 18.8%, and -21.7%, -13%, -10.4%. The year 2020 was marked by the COVID-19 pandemic, drastically affecting the global economy. While B&H, S&H, and Random Action benchmarks generated -25.3%, 12.6%, -18.8% returns in March, the ETDQN agent was smart enough to profit from the drop, resulting in outstanding 164% returns in a single month. In the following year, the strategy provides negative -13.7%, -14%, -15.9% returns, even though some positive 10.1%, 9.1%, and 8.8% balance the strategies' performance. Finally, during 2022, the strategy generates expressive 37.6%, 26.5%, and 23.7% returns. However, the policy generates -22.9%, -13.1%, and -11.8% negative returns in May, July, and November, respectively.

Unlike the previous assets, it is impossible to perform a deeper analysis since ATOM/USDT data frame contains only three years of data. Comparing the color overview generated by the heat map in Figure 16c, the map shows a positive-sided returns tendency. In 2019, TDQN outperformed all strategies. However, during the cryptocurrency bull run in 2020, B&H and TDQN beat the proposed algorithm. ETDQN outperformed all the previous benchmarks again in 2021, generating returns such as 72.4%, 163%, 116%, and 73.4%, subsequently in August, September, October, and November, concentrating expressive returns at the year's end.

Table 6

Annual returns in percentage generated by B&H, S&H, Random, ETDQN, TDQN strategies between 2009 to 2022 applied to iShares S&P500 ETF, Western Digital Corporation, and ATOM/USDT cryptocurrency pair assets.

ASSET / STRATEGY	iShares S&P500 ETF					Western Digital Corporation					ATOM/USDT cryptocurrency				
	B&H	S&H	Random	ETDQN	TDQN	B&H	S&H	Random	ETDQN	TDQN	B&H	S&H	Random	ETDQN	TDQN
2009	-0.56	0.56	0.31	0.56	3.04	X	X	X	X	X	X	X	X	X	X
2010	13.37	-14.6	-0.31	28.15	22.77	13.84	-14.09	-9.42	8.72	29.71	X	X	X	X	X
2011	-2.74	-2.28	-8.25	6.75	13.20	-9.35	-14.73	-7.41	3.38	67.36	X	X	X	X	X
2012	14.59	-14.31	6.91	3.45	10.47	35.89	-38.16	27.78	138.02	6.53	X	X	X	X	X
2013	28.47	-23.07	-3.55	-4.36	29.05	99.52	-53.81	-10.16	13.35	-47.08	X	X	X	X	X
2014	9.94	-9.99	10.58	-3.22	12.18	33.48	-29.34	-13.26	11.7	6.23	X	X	X	X	X
2015	-5.34	3.19	5.58	11.85	14.24	-45.52	63.67	-1.03	84.94	61.74	X	X	X	X	X
2016	13.94	-13.9	0.54	35.49	15.03	10.61	-29.82	-8.13	41.73	120.43	X	X	X	X	X
2017	12.99	-12.00	-1.67	10.95	19.22	17.62	-23.50	20.03	-4.21	-12.26	X	X	X	X	X
2018	-11.66	10.61	2.11	45.87	-17.20	-53.58	88.03	-23.73	187.24	39.17	X	X	X	X	X
2019	28.59	-23.51	0.56	11.11	29.42	71.93	-54.00	18.59	-18.66	-58.77	-15.97	-33.33	-81.8	253.45	-63.81
2020	-1.16	-10.27	-40.8	68.75	1.05	-13.55	-26.93	1.20	140.85	-22.57	30.28	-79.65	-82.57	27.73	348.30
2021	22.37	-19.65	-4.85	19.55	20.51	19.67	-31.63	-25.94	-26.8	10.25	462.68	-98.14	-46.3	646.56	-53.57
2022	-11.12	9.70	7.54	4.14	-0.27	-43.9	44.65	6.60	34.29	-30.63	X	X	X	X	X

Regarding the S&H benchmark, ETDQN outperforms it every year. S&H generated positive returns of 30.7% and 41.9% in 2019, but unfortunately, it also holds more expressive negative -33% -25.8% and -19.9% returns. Lastly, the proposed algorithm also outperformed the Random Action benchmark. During 2019, the policy had significant negative return amounts, such as -56.5%, -39.8%, and -21.8% in September, July, and June, respectively. The policy achieved its worst-performing month in 2020, generating significant -40.3%, -36.3%, and 29.4% negative returns. In 2021, the strategy generated a distinguished 66.7% return and some positive 52.6%, 33%, and 34.5% profits. Unfortunately, the policy cannot keep its gains and finishes the year with negative returns.

6.5. Annual Returns

This subsection analyzes annual returns generated by each benchmark to give the reader a better overview of each strategy's performance yearly. Table 6 shows annual returns from each benchmark and the developed algorithm strategy applied to iShares S&P500 ETF, Western Digital Corporation, ATOM/USDT data frames. In addition, the annual average return is also analyzed, representing the strategy's historical average return.

Concerning the iShares S&P500 ETF, TDQN had the best performance with 3.04% returns in 2009, followed by ETDQN with 0.56%. On the other hand, ETDQN was the most lucrative in 2010, beating TDQN by 5.98%. In the next five years, TDQN outperformed all benchmarks, and from 2016 to 2021, these two strategies kept alternating in the top performance position. Finally, in 2022 S&H takes the best-performing position. Even though TDQN had the first position for nine years and ETDQN for four years only, the second strategy performed better with respect to the AAR metric, which in this case, represents the historical arithmetic average return between the period of 2009 and 2022. The policy had an AAR of

17.07% against 12.34% produced by TDQN. ETDQN showed more aggressive behavior, delivering higher and more volatile returns with a 66.14% higher standard deviation than TDQN's.

In the Western Digital Corporation data frame, ETDQN took the third position regarding performance, having a difference of 20.99% and 5.12% returns from TDQN and B&H benchmarks in 2010. During the following year, the strategy took the second position staying behind TDQN with a 63.68% return. In 2012, ETDQN outperformed the remaining benchmarks, achieving a 138.02% return. In 2016, TDQN stayed in first position, causing approximately triple returns compared to ETDQN. However, the following year, it returned to the second position, losing to the B&H benchmark, which generated a 99.52% value. In 2014, the agent finished the year with approximately half of the returns produced by the B&H benchmark, but it took back the lead, generating 84.94% return in 2015. From 2017 onwards, TDQN cannot generalize well and produces few significant returns, such as in 2018 and 2021. During 2018 and 2020, ETDQN can take profit from both dumps and delivers its best 187.24% and 140.85% returns. The second one can be explained due to the COVID-19 pandemic. Lastly, the B&H strategy assumes the first performance evaluation position in 2019 and 2021, producing 71.93% and 19.67%, respectively, and in 2022 S&H outperforms ETDQN with a difference of 10.36%.

Even though both the B&H benchmark and ETDQN had the same number of times that achieved the first position concerning annual returns, ETDQN had an AAR from the year 2010 to 2022 of 47.27%, against 10.52%. In addition, the TDQN presented the AAR from 2010 to 2022 of 13.09%, being positioned in the second position regarding annual returns. Once again, ETDQN showed aggressive behavior with the highest standard deviation of 65.93, compared to 48.44 and 43.14 from TDQN and B&H benchmarks, respectively. According to the analysis, despite its high volatility, the model shows that the model tends to produce positive returns with high volatility, which is the reason that it achieved the best performance compared to all benchmarks.

In ATOM/USDT cryptocurrency data frame, ETDQN outperformed all benchmarks during 2019 and 2021, generating 253.45% and 646.56% annual returns. However, the strategy had the second position in 2020, losing to TDQN, which produced a 348.30% return. ETDQN had the best performance regarding AAR from 2019 to 2021, with 309.25%. The strategy again showed the most aggressive and volatile behavior, having 33.24% and 18.61% higher standard deviation than the TDQN and B&H benchmarks, respectively. By analyzing which position each benchmark achieved during every year concerning the annual returns, it

is possible to affirm that ETDQN always had the first or second position, even though the other benchmarks alternate between first, third, and last places.

7. Conclusion

This work proposed a DRL-based trading system that processes information according to market activity. The proposed system's main component is a DQN variant that extends the Trading DQN benchmark by incorporating enhancements that develop a policy that generalizes across multiple market scenarios, identifies trading opportunities, and takes advantage of them. ETDQN optimizes its decisions by receiving non-frequent feedback from the environment and replaying prioritized experiences containing different sub-goals, which assists in its primary goal of retaining the maximum profits. It is observable that, as opposed to the TDQN benchmark, the proposed algorithm could successfully generalize its policy and identify main events related to higher volatility, such as the COVID-19 dump during March 2020 and the ATOM/USDT pump during July 2021, taking profits from it, without the need of complex reward tuning.

Exponential profit and loss was the adopted reward function based on the assumption that the agent could notice an exponential growth rate during a particular time. A trading environment was created containing candlestick Dollar bars and RSI, MACD, Momentum, and Aroon oscillator technical indicators generated from it. In addition, it also included time signatures of the current hour and day of the week.

Historical intraday tick data from iShares Core S&P 500 ETF, Western Digital Corporation, and ATOM/USDT assets were collected and pre-processed according to market value to generate the previously mentioned bars. They do not undersample information during high market activity nor over-sample information during low activity. iShares Core S&P 500 ETF generated 398,362 Dollar bars using a 500K USD threshold, Western Digital Corporation developed 550,598 bars using a 1M USD threshold, and ATOM/USDT produced 359,786 intraday Dollar bars using a 7.5K USDT threshold.

Concerning daily mean cumulative returns, ETDQN proved to be approximately 1.46 and 7.13 times more lucrative than the second-best evaluated benchmark in the iShares S&P500 ETF and Western Digital Corporation data frames, TDQN. In addition, the same strategy was 2.14 times more lucrative than the second best-evaluated benchmark in iShares S&P500 ETF, B&H.

Regarding the mean 6-month rolling Sharpe ratio metric, the previously mentioned strategy outperformed every benchmark in both iShares S&P500 ETF and Western Digital Corporation data frames. The

algorithm achieved better 1.01 and 3.33 times risk-return relation compared to TDQN and B&H benchmarks. Unfortunately, even though the agent's developed strategy achieved a 7.3 maximum rolling Sharpe ratio in the ATOM/USDT, it did not outperform the B&H benchmark concerning the mean in this data frame.

Concerning maximum drawdown, ETDQN had 10.7% less risk of ruin in iShares S&P500 ETF than the second-best evaluated benchmark, TDQN strategy. In Western Digital Corporation, it had 8.8% less risk of ruin than the second-best considered benchmark, the random action strategy. However, in the ATOM/USDT data frame, the policy did have 2.7% more risk of ruin than the B&H benchmark.

Finally, the proposed strategy outperformed every benchmark regarding the AAR metric. In iShares S&P500 ETF, the ETDQN had an AAR of 17.07% against 12.34% produced by TDQN. In addition, it showed more aggressive behavior, delivering higher and more volatile returns with a 66.14% higher standard deviation than TDQN. Regarding the Western Digital Corporation, the same algorithm had an AAR from 2010 to 2022 of 47.27%, against 13.09% generated by TDQN from 2010 to 2022. Once again, it showed aggressive behavior with the highest standard deviation of 65.93, compared to 48.44 from TDQN. Finally, in ATOM/USDT data frame, the ETDQN had the best performance regarding AAR from 2019 to 2021 with 309.25%, showing the most aggressive and volatile behavior, having 33.24% and 18.61% higher standard deviation than the TDQN and B&H benchmarks.

This work considered a single position size only; hence the algorithm trades a single amount of the asset. Future works may extend it to a position size more significant than one. In addition, Sentiment analysis could also be explored to enhance the information in the states. Natural language processing techniques can be used to read texts and predict sentiment classes, serving as input to the state. As suggested by Ribeiro et al. (2021), including models to predict price return volatility may also provide information on upcoming opportunities. Instead of using only past and current prices in the state, computational intelligence approaches and linear models could also be incorporated to forecast upcoming prices (Santos et al., 2007).

It is difficult to guide an agent to its objective depending on how delayed its reward is provided. Temporal-difference learning creates bias. This is even more severe when rewards are sparse. Reward decomposition techniques could be explored, attributing compensation once a sub-goal is completed and maintaining the generalization capability.

Concerning distributional learning, the implemented approach uses discrete distributions, which have limitations. The number of atoms and bounds of the support requires domain knowledge of the task; hence a different approach could be explored to change the proposed parameterization.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

This research was funded by Conselho Nacional de Desenvolvimento Científico e Tecnológico – CNPq (Grants number: 307958/2019-1-PQ, 307966/2019-4-PQ, 404659/2016-0-Univ, 408164/2021-2-Univ), and PRONEX ‘Fundação Araucária’ 042/2018.

CRedit authorship contribution statement

Lucas de Azevedo Takara: Conceptualization, Methodology, Formal analysis, Validation, Writing - original draft, Writing - review editing.

André Alves Portela Santos: Conceptualization, Writing - review editing.

Leandro dos Santos Coelho: Conceptualization, Writing - review editing.

Viviana Cocco Mariani: Conceptualization, Writing - review editing.

References

- Andrychowicz, M., Crow, D., Ray, A., Schneider, J., Fong, R., Welinder, P., McGrew, B., Tobin, J., Abbeel, P., Zaremba, W., 2017. Hindsight experience replay, in: Advances in Neural Information Processing Systems 30: Annual Conference on Neural Information Processing Systems, Long Beach, USA. pp. 5048–5058.
- Arratia, A., 2014. Statistics of Financial Time Series. Atlantis Press, Paris. pp. 37–70.
- Bellemare, M.G., Dabney, W., Munos, R., 2017. A distributional perspective on reinforcement learning, in: Proceedings of the 34th International Conference on Machine Learning, PMLR, Sydney, Australia. pp. 449–458.
- CAPES, 2023. Capes periodical portals. <https://www.periodicos.capes.gov.br>. Accessed at 17/01/2023 (in Brazilian Portuguese).
- Carta, S., Corrigan, A., Ferreira, A., Podda, A.S., Recupero, D.R., 2021. A multi-layer and multi-ensemble stock trader using deep learning and deep reinforcement learning. Applied Intelligence 51, 889–905.
- Choi, J., 2021. Maximum drawdown, recovery, and momentum. Journal of Risk and Financial Management 14, 542.
- Conegundes, L., Pereira, A.C.M., 2020. Beating the stock market with a deep reinforcement learning day trading system, in: International Joint Conference on Neural Networks (IJCNN), Glasgow, UK. pp. 1–8.
- van Eck, N.J., Waltman, L., 2010. Software survey: Vosviewer, a computer program for bibliometric mapping. Scientometrics 84, 523–538.

- Fortunato, M., Azar, M.G., Piot, B., Menick, J., Hessel, M., Osband, I., Graves, A., Mnih, V., Munos, R., Hassabis, D., Pietquin, O., Blundell, C., Legg, S., 2018. Noisy networks for exploration, in: 6th International Conference on Learning Representations Conference Track Proceedings, OpenReview.net, Vancouver, Canada.
- Jiang, Z., Liang, J., 2017. Cryptocurrency portfolio management with deep reinforcement learning, in: Intelligent Systems Conference (IntelliSys), London, UK. pp. 905–913.
- Kissell, R.L., 2021. Algorithmic Trading Methods. 2nd ed., Academic Press, Cambridge, Massachusetts, USA.
- Kitchenham, B., Brereton, P., 2013. A systematic review of systematic review process research in software engineering. *Information and Software Technology* 55, 2049–2075.
- Koratamaddi, P., Wadhwani, K., Gupta, M., Sanjeevi, S.G., 2021. Market sentiment-aware deep reinforcement learning approach for stock portfolio allocation. *Engineering Science and Technology, an International Journal* 24, 848 – 859.
- Levine, S., Finn, C., Darrell, T., Abbeel, P., 2016. End-to-end training of deep visuomotor policies. *Journal of Machine Learning Research* 17, 1334–1373.
- Li, Y., Zheng, W., Zheng, Z., 2019. Deep robust reinforcement learning for practical algorithmic trading. *IEEE Access* 7, 108014–108022.
- Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A.A., Veness, J., Bellemare, M.G., Graves, A., Riedmiller, M., Fiedjeland, A.K., Ostrovski, G., Petersen, S., Beattie, C., Sadik, A., Antonoglou, I., King, H., Kumaran, D., Wierstra, D., Legg, S., Hassabis, D., 2015. Human-level control through deep reinforcement learning. *Nature* 518, 529–533.
- de Prado, M.L., 2018. *Advances in Financial Machine Learning*. 1st ed., Wiley Publishing, Hoboken, New Jersey, USA.
- Ribeiro, G.T., Santos, A.A.P., Mariani, V.C., Coelho, L.S., 2021. Novel hybrid model based on echo state neural network applied to the prediction of stock price return volatility. *Expert Systems with Applications* 184, 115490.
- Sagiraju, K., Mogalla, S., 2022. Deployment of deep reinforcement learning and market sentiment aware strategies in automated stock market prediction. *International Journal of Engineering Trends and Technology* 70, 43 – 53.
- Santos, A.A.P., Costa, N.C.A., Coelho, L.S., 2007. Computational intelligence approaches and linear models in case studies of forecasting exchange rates. *Expert Systems with Applications* 33, 816–823.
- Schaul, T., Quan, J., Antonoglou, I., Silver, D., 2016. Prioritized experience replay, in: 4th International Conference on Learning Representations Conference Track Proceedings, San Juan, Puerto Rico.
- Sharpe, W.F., 1994. The sharpe ratio. *The Journal of Portfolio Management* 21, 49–58.
- Shavandi, A., Khedmati, M., 2022. A multi-agent deep reinforcement learning framework for algorithmic trading in financial markets. *Expert Systems with Applications* 208, 118124.
- Shin, H.G., Ra, I., Choi, Y.H., 2019. A deep multimodal reinforcement learning system combined with cnn and lstm for stock trading, in: International Conference on Information and Communication Technology Convergence (ICTC), Jeju Island, Korea. pp. 7–11.
- Si, W., Li, J., Ding, P., Rao, R., 2017. A multi-objective deep reinforcement learning approach for stock index future's intraday trading, in: 10th International Symposium on Computational Intelligence and Design (ISCID), Hangzhou, China. pp. 431–436.
- Théate, T., Ernst, D., 2021. An application of deep reinforcement learning to algorithmic trading. *Expert Systems with Applications* 173, 114632.
- Wang, Z., Schaul, T., Hessel, M., van Hasselt, H., Lanctot, M., de Freitas, N., 2016. Dueling network architectures for deep reinforcement learning, in: Proceedings of the 33rd International Conference on Machine Learning, ICML, JMLR.org, New York, USA. pp. 1995–2003.
- Wu, X., Chen, H., Wang, J., Troiano, L., Loia, V., Fujita, H., 2020. Adaptive stock trading strategies with deep reinforcement learning methods. *Information Sciences* 538, 142–158.
- Yang, H., Liu, X.Y., Zhong, S., Walid, A., 2020. Deep reinforcement learning for automated stock trading: An ensemble strategy, in: Proceedings of the 1st ACM International Conference on AI in Finance, Association for Computing Machinery, New York, NY, USA.